

Graphical Abstract

Cost-Sensitive Online Learning for Control Chart Pattern Recognition

Highlights

Cost-Sensitive Online Learning for Control Chart Pattern Recognition

- Cost-sensitive online models improve abnormal pattern detection in CCPR tasks.
- We introduced CSPA and CSOGD to adapt to evolving data with class-specific penalties.
- We showed its advantage over standard models on synthetic and real datasets.
- We compared the models' performance to detect different control chart pattern trends.

Cost-Sensitive Online Learning for Control Chart Pattern Recognition

Abstract

An essential requirement of modern quality control systems is to detect, in real time, abnormal control chart patterns that may lead to process disruptions and associated restoration costs if remain undetected. Traditional batch learning models often fall short in dynamic environments due to their inability to adapt to evolving data streams. This paper introduces a cost-sensitive online learning framework tailored for control chart pattern recognition under class-imbalanced environments, where abnormal patterns are rare but critical to detect. Leveraging the strengths of Online Gradient Descent and Passive-Aggressive algorithms, we propose novel cost-sensitive variants, CSPA, that incorporate class-specific penalties and surrogate loss functions to enhance detection of minority-class anomalies. Unlike prior approaches, our models continuously update with incoming data, enabling timely and robust recognition of evolving control chart patterns. We study the performance of our proposed models using both synthetic and real datasets. Our extensive experimental study indicates the superiority of our proposed online cost-sensitive framework over standard online learning algorithms as it achieves significantly higher sensitivity and overall classification performance, particularly in scenarios with short observation windows and subtle pattern deviations.

Keywords: Online learning, control chart pattern recognition, imbalanced data, cost-sensitive classification

1. Introduction

Control charts, introduced by W.A. Shewhart [1], are foundational tools in statistical process control. They facilitate monitoring of production processes by distinguishing normal process variation from deviations caused by specific issues, thereby helping ensure quality and operational stability [2]. By tracking critical process variables over time, control charts enable the detection of anomalies when a process deviates from its expected behavior, thus allowing the manufacturers to make informed decisions to optimize resource utilization and maintain standardized outcomes. Consistent use of control charts enables manufacturers to achieve tight process control and optimize economic outcomes [3]. Distinct control chart patterns (CCPs) include normal, up-trend (UT), down-trend (DT), cyclic (CYC), systematic (SYS), stratification (STR), up-shift (US), and down-shift (DS); see Figure 1 (b–h). Up-trend and down-trend patterns typically result from gradual equipment wear or the degradation of equipment, while shift patterns, on the other hand, represent abrupt deviations, such as material variations or unforeseen equipment failures [4]. Cyclic patterns sometimes indicate periodic variations influenced by environmental factors or alternating operators, while systematic patterns reveal recurring deviations, potentially due to persistent inaccuracies in

measurement tools or process parameters [5, 6]. Early detection of distinct CCPs, regardless of their type, is crucial for preventing significant disruptions and improving operational efficiency [7]. This is particularly becoming challenging when the patterns must be detected in today’s complex manufacturing processes, making control chart pattern recognition (CCPR) even more valuable for the automatic identification of process anomalies.

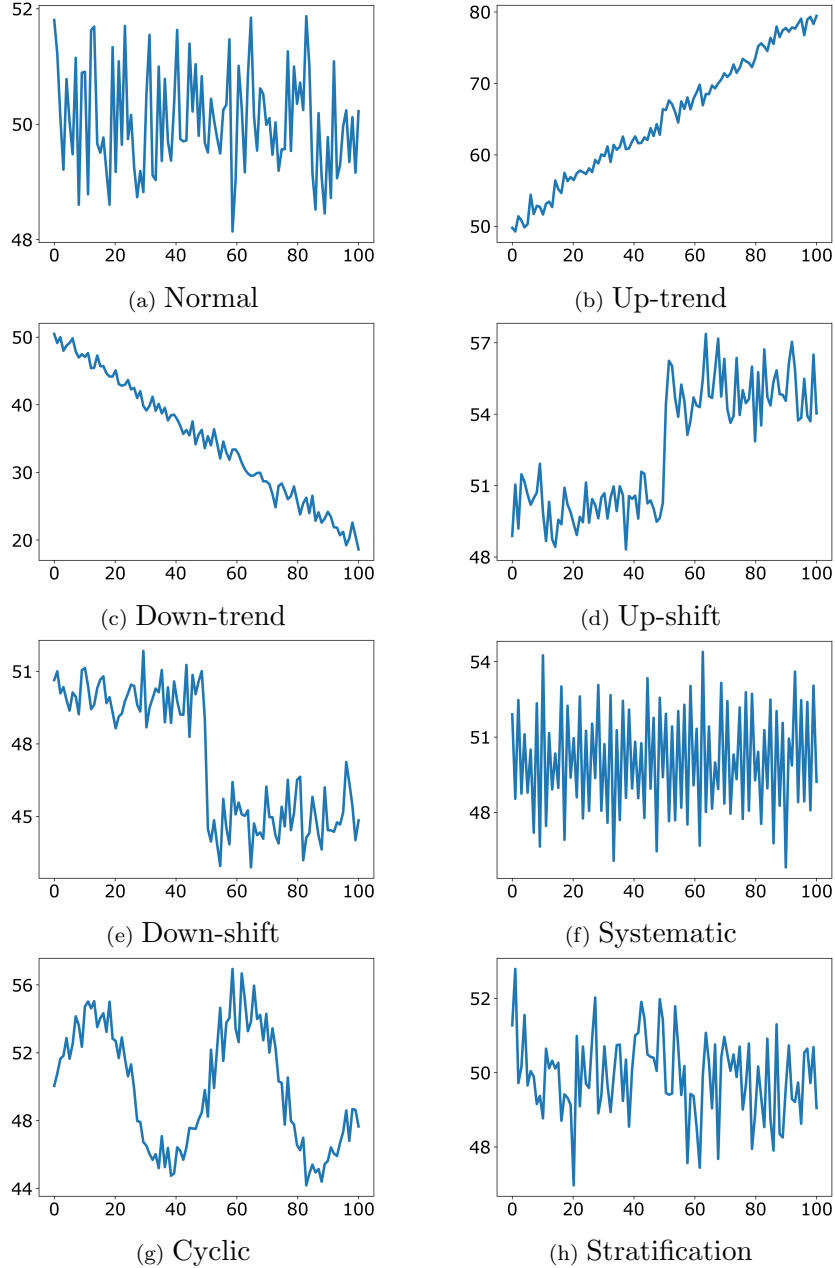


Figure 1: Examples of common CCPs

Manual CCPR relies heavily on human expertise and require extensive training to monitor CCP and identify abnormal patterns that signal process irregularities. This is a very time-consuming process, is subject to human error, and falls short in identifying the more complex patterns [8]. To address this challenge, machine learning techniques have shown promise in

enhancing CCPR accuracy [9]. This ranges from supervised learning-based models that are trained on historical, labeled control charts [10, 11, 12, 13, 14, 15, 16], to unsupervised methods that can identify anomalies when the labels of the CCPs are not available [17, 18]. By addressing the limitations of manual CCPR methods, machine learning-based models offer high accuracy and have been proved to be scalable to settings with massive-size historical data, making them an essential decision-aid tool for early fault detection [9].

Most machine learning-based CCPR framework are essentially offline models (also known as batch learning) in which the model is trained once on the historical datasets obtained from the process. A major limitation of such models is their inability to adapt to new evolving patterns that have not been captured within historical data. Although it is possible to update the model by retraining with new samples, this procedure is computationally expensive and impractical in rapidly changing manufacturing environments. To overcome this limitation, online learning has become a more effective and scalable approach for pattern detection in such settings. In contrast to batch learning models that must be retrained on the entire historical dataset, online learning algorithms update their parameters incrementally with each incoming data sample [19]. This continuous adaptation makes online learning particularly well-suited for CCPR in manufacturing settings with evolving control patterns.

Despite its potential, the literature on online learning for CCPR remains limited [15, 20, 21, 22]. Yang and Zhou [20] have developed an online CCPR system using a neural network ensemble model, but their approach did not address the bias in the correlation coefficient introduced by abnormal patterns. Consequently, they trained a separate CCPR model for each assumed autocorrelation level [9]. An SVM-based method was proposed by Lin et al. [21] to detect seven common abnormal patterns online, with the assumption that process observations follow a first-order autoregressive model. Furthermore, Du et al. [22] employed a wavelet transform in combination with a particle swarm-optimized SVM model to enable online recognition of concurrent CCPs. However, prior research has shown that directly applying standard SVM models to CCPR tasks often results in poor performance due to inherent class imbalance issues [8]. Moreover, these studies often do not use online learning methods for training and testing their models [6].

Several online learning techniques have been widely studied within the machine learning community [19, 23, 24]. Among them, Online Gradient Descent (OGD) [25] and Passive-Aggressive (PA) learning [19] have been successfully applied to real-time decision-making tasks due to their sub-linear regret bounds and computational efficiency [26, 27, 28, 29, 30]. The PA algorithm, in particular, is well-suited for handling noisy data and supports incremental updates, which makes it robust in scenarios where the underlying data distribution may shift over time [19]. However, most existing online learning methods often face significant challenges when applied to highly imbalanced datasets, where one class substantially outnumbers the others [31]. If left untreated, class imbalance can lead to biased models toward the majority class, which may fall short in true detection of the minority class. This is particularly relevant to CCPR models as abnormal patterns are rare but critical to detect, and misclassification can lead to serious consequences. For example, false negatives, failing to detect a true process failure, are typically more costly than false positives. This asymmetry in misclassification costs reveals the need for effective strategies to handle class

imbalance in industrial environments, where early detection is crucial to minimizing risk and operational costs.

Cost-sensitive learning addresses this challenge by explicitly incorporating a cost matrix or class weights to penalize errors differently, thereby reducing the model’s bias toward the majority class in imbalanced datasets [8, 32]. Its online counterpart extends this approach to real-time applications by continuously minimizing weighted classification errors, thereby improving the timely detection of rare but critical abnormal patterns [32]. Previous studies have demonstrated the effectiveness of weighted SVMs for CCPR tasks [8]; however, their reliance on batch learning limits their adaptability in dynamic, real-time environments. Although the cost-sensitive OGD (CSOGD) algorithm [32] has been introduced for online classification problems, it has not yet been applied to CCPR, representing a significant gap in the current literature. This gap is particularly notable given that the cost-sensitive learning field has advanced significantly in recent years, introducing adaptive [33], sparse [34, 35], and kernel-based [36] variants to handle complex, high-dimensional data. Additionally, while the online PA algorithm shares theoretical foundations with SVMs, it has not been extended to a cost-sensitive CCPR framework.

The main contributions of this work are summarized as follows.

- We introduce cost-sensitive online prediction models for CCPR, including variants of Passive-Aggressive (CSPA) and Cost-Sensitive Online Gradient Descent (CSOGD). Unlike traditional online learning methods that assume balanced misclassification costs, our models explicitly incorporate asymmetric penalties to reflect the unequal consequences of false negatives and false positives in CCPR settings. To the best of our knowledge, this is the first study to develop cost-sensitive online learning methods tailored specifically to the CCPR problem.
- Our proposed CSPA models incorporate class-specific penalty terms and surrogate loss functions to effectively address class imbalance. By systematically incorporating cost-sensitive weighting schemes into both the update rules and optimization objectives, we derive eight new algorithmic variants. These formulations provide flexible mechanisms to control margin updates and adapt to varying imbalance levels, which can improve robustness in rare-event detection scenarios.
- We derive closed-form update rules for all eight CSPA algorithms by solving the corresponding constrained optimization problems using the Karush-Kuhn-Tucker (KKT) conditions. The resulting algorithms have a time complexity of $O(mT)$, where T is the number of instances and m is the feature dimension. This indicates the efficiency of our proposed algorithm for real-time CCPR in manufacturing environments.
- We provide a comprehensive evaluation of the proposed CSPA and CSOGD models in comparison against standard online learning methods and four state-of-the-art cost-sensitive online learning algorithms under multiple experimental conditions, including diverse abnormal pattern types, varying sliding window lengths, and different imbalance levels. In addition to extensive simulation experiments, we further validate their practical value by applying them to a real-world pharmaceutical dataset demonstrating their effectiveness in dynamic and highly imbalanced environments.

The organization of this paper is as follows. Section 2 reviews the related work on cost-sensitive learning. The online learning framework is introduced in Section 3.1, followed by detailed explanation of standard OGD and PA models in Sections 3.2 and 3.3, respectively. Section 3.4 details the proposed CSOGD algorithm, while the proposed CSPA algorithm is discussed in Section 3.5. We outline the performance metrics used to study our proposed algorithms in Section 3.6, and conclude the second section with an overview of our synthetic data generation scheme in Section 3.7. We present the results of our proposed models on both synthetic and real datasets in Section 4. We provide further insights and suggest future extensions of our work in Section 5.

Remark. The mathematical notations used in this paper follow those introduced in the author’s earlier thesis work [37].

2. Related Work

This section provides an overview of state-of-the-art techniques in cost-sensitive learning and online classification that are most relevant to our work. The foundation of cost-sensitive online learning lies in explicitly accounting for class imbalance during model optimization. This is typically achieved by formulating one of two training objectives: (1) minimizing a cost-sensitive loss function, where higher penalties are assigned to misclassifications of minority-class samples, or (2) maximizing a cost-weighted performance metric, such as a weighted sum of sensitivity and specificity. Wang et al. (2013) [32] pioneered this direction with the CSOGD framework, which directly modifies the update rules of standard OGD to penalize misclassifications of the minority class. Building on this, Zhao et al. (2018) [33] introduced the Adaptive Regularized Cost-Sensitive Online Gradient Descent (ACOG) algorithm, which incorporates second-order information to improve convergence stability and robustness.

To handle high-dimensional streaming data, recent research has focused on integrating sparsity-inducing regularization into cost-sensitive frameworks. Examples of such methods include the Cost-Sensitive Truncated Gradient (CSTG) [34] and Cost-Sensitive Regularized Dual Averaging (CSRDA) [35] algorithms, which simultaneously minimize classification cost while promoting model sparsity, thereby enabling effective feature selection in real-time environments. The CSRDA algorithm extends the Regularized Dual Averaging approach [38] by embedding cost-sensitive mechanisms through ℓ_1 -norm regularization and dual averaging updates, making it well-suited for imbalanced and high-dimensional data streams. Chen et al. (2023) [39] further extended this framework by formulating a convex optimization function that directly optimizes four ℓ_1 -regularized, cost-sensitive objective functions. They established theoretical regret bounds of $O(\sqrt{T})$, guaranteeing convergence with respect to both classification cost and sparsity in severely imbalanced, high-dimensional streaming data. Building on dual averaging techniques, Chen et al. (2022) [40] subsequently proposed the Proximal Cost-Sensitive Sparse Group Online Learning (PCSGOL) framework, which integrates group-wise sparsity via combined ℓ_2 - and ℓ_1 -norm regularization to exploit inherent group structures within high-dimensional features in imbalanced online learning settings. In contrast, CSTG builds upon the Truncated Gradient method [41], formulating a convex

optimization problem that balances misclassification cost and model sparsity via truncated gradient updates.

Beyond linear models, recent studies have introduced advanced methods to handle non-linearity and concept drift in imbalanced streams. Chen et al. (2023) [36] proposed a Cost-Sensitive Online Adaptive Kernel (COAK) learning algorithm, which leverages random feature mapping to approximate kernel functions in an online setting, thereby enabling the classification of large-scale non-linear data. Extending kernel-based approaches to handle concept drift in dynamic environments, Chen et al. (2024) [42] have developed the Cost-Sensitive Continuous Ensemble Kernel Learning (CCEKL) method. CCEKL introduces an adaptive misclassification cost function that adjusts to evolving imbalance rates and employs parallel ensemble training with multiple kernel widths to maintain classification performance as data distributions shift over time. To tackle temporal evolution and imbalance shifts, Usman and Chen (2024) [43] have developed the Intensive Class Imbalance Learning (ICIL) framework, which employs feature-sensitive detectors to identify virtual concept drifts and dynamically calibrate the model against changing imbalance ratios. Most recently, alternative learning architectures beyond standard gradient-based methods have been emerged for imbalanced classification. Li et al. (2023) [44] have introduced the Adaptive Weighted Imbalanced Least Squares Regression (AWILSR) model, which combines adaptive weight learning with label relaxation to better capture minority class distributions. Similarly, they have proposed the Complemented Subspace-based Weighted Collaborative Representation (CSCR-W) method [45], which integrates adaptive class weights into collaborative representation classifiers to reduce majority class bias. Furthermore, extending into neural architectures, Li et al. (2025) [46] developed an Imbalanced Broad Learning System (LSIBLS) that combines genetic programming for feature optimization with a latent label space, demonstrating that structural adaptations in learning objective can significantly enhance robustness under extreme imbalance.

3. Methodology

3.1. Online Learning

Online learning refers to a learning framework in which the predictive model is updated as new samples are incrementally added to the training set. This is performed by continuously adapting the model parameters without the need for re-training the model based on all prior data samples. This is in contrast to traditional learning algorithms, which build models using the entire dataset at once. Online learning enhances the accuracy of predictions by leveraging the correct outcomes from prior tasks and relevant information, which is particularly beneficial for applications involving continuous data streams [47].

When a new sample $\mathbf{x}_t$ is received at time step t , online learning starts by making the prediction $\hat{y}_t = \text{sign}(\omega_t^\top \mathbf{x}_t)$, where ω_t indicates the current model parameters computed in the prior training. With the true label y_t in hand, the prediction loss is computed. If the loss is greater than zero, the algorithm updates the classifier’s parameters from ω_t to ω_{t+1} . This is repeated incrementally until the entire sequence is received. The aim is to minimize the total number of prediction errors in T rounds of incremental re-training steps [29, 48],

denoted by M_t and defined as

$$M_T = \sum_{t=1}^T \mathbb{I}(\hat{y}_t \neq y_t) \quad (1)$$

where the indicator function $\mathbb{I}(z)$ returns 1 if z is true, and 0 otherwise. The pseudo-code for the online learning framework for binary classification is provided in Algorithm 1.

Algorithm 1 Online Binary Classification Learning [29]

- 1: Initialize the weight ω_1 ;
 - 2: **for** $t = 1, \dots, T$ **do**
 - 3: Receive input example $\mathbf{x}_t \in \mathbb{R}^m$;
 - 4: Predict $\hat{y}_t = \text{sign}(\omega_t^T \mathbf{x}_t)$ as the label of x_t ;
 - 5: Receive the true label $y_t \in \{-1, +1\}$;
 - 6: Compute loss $\ell(\omega_t; (\mathbf{x}_t, y_t))$;
 - 7: Update the model weight ω_t to ω_{t+1} based on the loss;
 - 8: **end for**
-

Online learning methods have extensively been explored in the literature [29], ranging from unified frameworks for sparse online learning [49] to cost-sensitive learning algorithms designed to address concept drift and extreme class imbalance [43, 50]. In this study, we focus on two prominent approaches: PA learning [19] and OGD [25]. These methods belong to the family of first-order optimization techniques, which utilize first-order (sub)gradient information to minimize objective functions. A detailed discussion of these methods is provided in the following sections.

3.2. OGD

The OGD algorithm, originally proposed by [25], is a widely adopted and easy-to-implement approach for solving convex optimization problems in online settings. As the online counterpart to stochastic gradient descent [29], OGD similarly updates the model by taking a step in the direction opposite to the gradient of the loss function evaluated at the current instance $\mathbf{x}_t$ as

$$\omega_{t+1} = \omega_t - \lambda_t \nabla \ell(\omega_t; (\mathbf{x}_t, y_t)), \quad (2)$$

where λ_t is the step size at time t and ℓ indicates the loss function. A typical choice of the loss function for OGD is the hinge loss, which is commonly used for binary classification tasks [19, 32], defined as

$$\ell(\omega_t; (\mathbf{x}_t, y_t)) = \max\{0, 1 - y_t(\omega_t \cdot \mathbf{x}_t)\}. \quad (3)$$

3.3. PA Learning

PA learning [19] is a class of online learning algorithms that incrementally updates model parameters as new data becomes available. Unlike traditional first-order online learning methods, such as the well-known Perceptron algorithm [51], PA learning imposes a margin constraint to ensure that the prior instances, which had been correctly predicted, remain correctly classified and also lie at a minimum distance from the decision boundary [32, 52].

This margin-based approach improves the robustness of the classifier, particularly in dynamic and non-stationary environments. Specifically, each iteration of the PA algorithm involves solving the following constrained optimization problem:

$$(PA) \quad \omega_{t+1} = \arg \min_{\omega \in \mathbb{R}^m} \frac{1}{2} \|\omega - \omega_t\|^2 \quad \text{subject to} \quad \ell(\omega; (\mathbf{x}_t, y_t)) = 0, \quad (4)$$

which has the following closed-form solution

$$\omega_{t+1} = \omega_t + \tau_t y_t \mathbf{x}_t, \quad \tau_t = \frac{\ell(\omega_t; (\mathbf{x}_t, y_t))}{\|\mathbf{x}_t\|^2}$$

The hinge loss of Eq. 3 remains a common choice for the loss function of PA algorithms. To improve the robustness of the classifier with respect to noisy data, the hard constraint in Problem 4 is converted into a soft-margin constraint by introducing a slack variable ξ . To avoid a large loss value that may occur in training the classifier with sample $\mathbf{x}_t$, the slack variable is then penalized, using a positive penalty denoted by $C > 0$, in the objective function. Accordingly, two variants are introduced in the literature [19]:

$$(PA_1) \quad \omega_{t+1} = \arg \min_{\omega \in \mathbb{R}^m} \frac{1}{2} \|\omega - \omega_t\|^2 + C\xi \quad \text{subject to} \quad \ell(\omega; (\mathbf{x}_t, y_t)) \leq \xi, \quad (5)$$

with the following closed-form solution

$$\omega_{t+1} = \omega_t + \tau_t y_t \mathbf{x}_t, \quad \tau_t = \min \left\{ C, \frac{\ell(\omega_t; (\mathbf{x}_t, y_t))}{\|\mathbf{x}_t\|^2} \right\},$$

and

$$(PA_2) \quad \omega_{t+1} = \arg \min_{\omega \in \mathbb{R}^m} \frac{1}{2} \|\omega - \omega_t\|^2 + C\xi^2 \quad \text{subject to} \quad \ell(\omega; (\mathbf{x}_t, y_t)) \leq \xi, \quad (6)$$

with the following model update rule

$$\omega_{t+1} = \omega_t + \tau_t y_t \mathbf{x}_t, \quad \tau_t = \frac{\ell(\omega_t; (\mathbf{x}_t, y_t))}{\|\mathbf{x}_t\|^2 + \frac{1}{2C}}.$$

Remark. The choice hinge loss for ℓ (which is always non-negative) indicates that negative values of ξ are not feasible. In general, however, the constraint $\xi \geq 0$ is often imposed to soft-margin classifiers.

3.4. Cost-Sensitive OGD for CCPR

Cost-sensitive learning methods aim at treating the class imbalance by introducing class-specific misclassification penalties, where minority class errors incur higher costs than majority class errors [44, 45]. This is commonly performed by considering one of the following two objectives in the training task: 1) minimization of a cost function, where a higher penalty is applied to misclassification of samples in from the minority class, and 2) maximizing a weighted sum of sensitivity and specificity to offset the misleading effect of the majority class

in improving the accuracy of the trained model, which otherwise produces a classifier biased toward the majority class. However, as shown by Wang et al. [32], there exists a common form for the two objectives, which is represented as

$$\sum_{y_t=+1} \rho \cdot \mathbb{I}_{(y_t \omega \cdot \mathbf{x}_t < 0)} + \sum_{y_t=-1} \mathbb{I}_{(y_t \omega \cdot \mathbf{x}_t < 0)}, \quad (7)$$

where $\mathbb{I}(\cdot)$ is the indicator function, and the choice of ρ determines the type of the objective function. Specifically, Eq. (7) is equivalent to the objective of maximizing the weighted sum of specificity and sensitivity if $\rho = \frac{\eta_p T_n}{\eta_n T_p}$, where η_p and η_n represents the weights assigned for sensitivity and specificity, respectively, and T_p and T_n denote the size of the majority and minority classes in the sample, respectively. Further, Eq. (7) is equivalent to the objective of minimizing the weighted cost function if $\rho = \frac{c_p}{c_n}$, where c_p and c_n indicate the false-negative and false-positive costs, respectively.

Using Eq. (7), Wang et al. [32] proposed CSOGD as an extension of the OGD algorithm. They address the inherent non-convexity of Eq (7) by replacing the indicator function with convenient convex surrogate functions. Specifically, they propose two loss functions as

$$\ell^I(\omega_t; (\mathbf{x}_t, y_t)) = \max(0, (\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)}) - y_t(\omega_t \cdot \mathbf{x}_t)), \quad (8)$$

together with the following update rule:

$$\omega_{t+1} = \begin{cases} \omega_t + \lambda y_t x_t & \text{if } \ell^I(\omega_t) > 0 \\ \omega_t & \text{Otherwise} \end{cases},$$

and

$$\ell^{II}(\omega_t; (\mathbf{x}_t, y_t)) = (\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)}) * \max(0, 1 - y_t(\omega_t \cdot \mathbf{x}_t)), \quad (9)$$

together with

$$\omega_{t+1} = \begin{cases} \omega_t + \lambda \rho_t y_t x_t & \text{if } \ell^{II}(\omega_t) > 0 \\ \omega_t & \text{Otherwise,} \end{cases}, \quad \rho_t = \rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)}$$

The CSOGD algorithm corresponding to (8) is referred to as “CSOGD-I,” while the resulting algorithm corresponding to (9) is denoted by “CSOGD-II” [32]. Algorithm 2 provides the pseudocode for CSOGD-I and CSOGD-II. We note that the CSOGD algorithms are computationally efficient, with a time complexity of $O(T \times m)$, where T represents the number of instances and m denotes the dimensionality of the data.

Algorithm 2 CSOGD Algorithms [32]

Input: Learning rate λ ; bias parameter $\rho = \frac{\eta_p T_n}{\eta_n T_p}$ for “sum” and $\rho = \frac{c_p}{c_n}$ for “cost”

Output: Final weight vector ω_{T+1}

```

1: Initialize the weight vector  $\omega_1 = \mathbf{0}$ ;
2: for  $t = 1, \dots, T$  do                                ▷ Iterate over each data sample
3:   Receive instance  $\mathbf{x}_t \in \mathbb{R}^m$ ;
4:   Predict:  $\hat{y}_t = \text{sign}(\omega_t^\top \mathbf{x}_t)$ ;
5:   Receive correct label:  $y_t \in \{-1, +1\}$ ;
6:   Compute loss:  $\ell(\omega_t) = \ell^*(\omega_t; (\mathbf{x}_t, y_t))$ ;  $*$   $\in \{I, II\}$ ;
7:   if  $\ell(\omega_t) > 0$  then                                ▷ Update only if loss is positive
8:     Update the weight vector:  $\omega_{t+1} = \omega_t - \lambda \nabla \ell(\omega_t)$ ;
9:   else
10:     $\omega_{t+1} = \omega_t$ ;                                ▷ No update if loss is zero
11:   end if
12: end for
13: return  $\omega_{T+1}$ 

```

3.5. Cost-sensitive PA for CCPR

Cost-sensitive PA (CSPA), the cost-sensitive variant of the PA algorithm, is designed to address class imbalance problem. Crammer et al. [19] developed a cost-sensitive extension of the PA algorithm for multi-class classification problems; however, their proposed approach does not directly optimize a cost-sensitive measure [29]. Instead, while incorporating cost-sensitivity aspects, it adopts an update process that follows an optimization formulation similar to that of standard PA learning. This approach may be less effective in cost-sensitive applications requiring direct measure optimization.

To address this issue, we propose a new cost-sensitive PA variant, namely CSPA₁ and CSPA₂, which incorporates class-specific weights into the PA₁ and PA₂ formulations, respectively, to ensure that instances from the minority class receive higher importance in learning. Consequently, the weight update rules for the CSPA₁ and CSPA₂ are adjusted to apply label-specific penalties. The aggressiveness parameter associated with the positive (C^+) and negative (C^-) classes are defined as:

$$C^+ = \frac{C}{T_p}, \quad C^- = \frac{C}{T_n},$$

$$C_t = \begin{cases} C^+ & \text{if } y_t = +1, \\ C^- & \text{if } y_t = -1. \end{cases} \quad (10)$$

where T_p and T_n denote the size of the positive (abnormal) and negative (normal) classes, respectively. The aggressiveness parameter C_t dynamically scales the step size τ_t based on the misclassification cost associated with the class of the current data instance. The optimization problem for CSPA₁ is formulated as follows:

$$\omega_{t+1} = \arg \min_{\omega \in \mathbb{R}^m} \frac{1}{2} \|\omega - \omega_t\|^2 + C_t \xi \quad \text{subject to} \quad \ell(\omega; (\mathbf{x}_t, y_t)) \leq \xi, \quad \xi \geq 0, \quad (11)$$

By solving the Lagrangian of the above optimization problem and satisfying the KKT conditions, one can derive the closed-form update rules as follows:

$$\tau_t = \min \left\{ C_t, \frac{\max(0, 1 - y_t(\omega_t \cdot \mathbf{x}_t))}{\|\mathbf{x}_t\|^2} \right\} = \min \left\{ C_t, \frac{\ell(\omega_t; (\mathbf{x}_t, y_t))}{\|\mathbf{x}_t\|^2} \right\}. \quad (12)$$

The optimization problem for CSPA₂ is formulated as follows:

$$\omega_{t+1} = \arg \min_{\omega \in \mathbb{R}^m} \frac{1}{2} \|\omega - \omega_t\|^2 + C_t \xi^2 \quad \text{subject to} \quad \ell(\omega; (\mathbf{x}_t, y_t)) \leq \xi, \quad \xi \geq 0, \quad (13)$$

The closed-form update rule can be derived by solving the Lagrangian of the above optimization problem and satisfying the KKT conditions as follows:

$$\tau_t = \frac{\max(0, 1 - y_t(\omega_t \cdot \mathbf{x}_t))}{\|\mathbf{x}_t\|^2 + \frac{1}{2C_t}} = \frac{\ell(\omega_t; (\mathbf{x}_t, y_t))}{\|\mathbf{x}_t\|^2 + \frac{1}{2C_t}}. \quad (14)$$

In addition to the CSPA₁ and CSPA₂ algorithms, we introduce six new variants to further improve cost-sensitive classification performance in CCPR tasks. These variants replace the standard hinge loss, used in the standard PA, PA₁, and PA₂ algorithms [19], with the modified hinge loss functions ℓ^I and ℓ^{II} , as defined in Eqs. 8 and 9. The resulting algorithms are denoted as CSPA- ℓ^I , CSPA- ℓ^{II} , CSPA₁- ℓ^I , CSPA₁- ℓ^{II} , CSPA₂- ℓ^I , and CSPA₂- ℓ^{II} . We hypothesize that substituting the original hinge loss with ℓ^I and ℓ^{II} will enhance the performance of CSPA algorithms in highly imbalanced datasets, where reducing the impact of costly misclassifications is crucial. Similar to CSPA₁ and CSPA₂, the newly proposed variants, excluding those based on the standard PA algorithm, incorporate class-specific penalty terms C_t (as defined in Eq. 10) and solve the PA optimization equations using the ℓ^I and ℓ^{II} loss functions. These loss functions are utilized to compute the updated weights and step size τ_t , as detailed in the following sections.

3.5.1. CSPA- ℓ^I

CSPA- ℓ^I incorporates the modified loss function ℓ^I into the standard PA algorithm (Eq. 4), and is formulated as follows:

$$\omega_{t+1} = \arg \min_{\omega \in \mathbb{R}^m} \frac{1}{2} \|\omega - \omega_t\|^2 \quad \text{subject to} \quad \ell^I(\omega; (\mathbf{x}_t, y_t)) = 0, \quad (15)$$

The Lagrangian of the above optimization problem is formulated as:

$$\mathcal{L}(\omega, \tau_t, \lambda) = \frac{1}{2} \|\omega - \omega_t\|^2 + \tau_t \left(\max \left(0, \left(\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)} \right) - y_t(\omega \cdot \mathbf{x}_t) \right) \right),$$

By calculating the derivatives of $\mathcal{L}$ with respect to ω and setting it to zero, we obtain the closed-form update rules for the CSPA- ℓ^I algorithm:

$$\frac{\partial \mathcal{L}}{\partial \omega} = \omega - \omega_t - \tau_t y_t \mathbf{x}_t = 0,$$

$$\omega = \omega_t + \tau_t y_t \mathbf{x}_t,$$

$$\tau_t = \frac{(\rho * I_{(y_t=1)} + I_{(y_t=-1)}) - y_t(\omega_t \cdot \mathbf{x}_t)}{\|\mathbf{x}_t\|^2} = \frac{\ell^I(\omega_t; (\mathbf{x}_t, y_t))}{\|\mathbf{x}_t\|^2}. \quad (16)$$

The derivations of the CSPA- ℓ^I update rules are presented in Appendix A.1.

3.5.2. CSPA₁- ℓ^I

CSPA₁- ℓ^I is an extended variant of CSPA₁ that incorporates the ℓ^I loss function. The Lagrangian corresponding to the optimization problem in Eq. 11, formulated using the ℓ^I loss, is given by:

$$\mathcal{L}(\omega, \xi, \tau_t, \kappa) = \frac{1}{2} \|\omega - \omega_t\|^2 + C_t \xi + \tau_t [\ell^I(\omega; (\mathbf{x}_t, y_t)) - \xi] - \kappa(\xi), \quad (17)$$

where $\tau \geq 0$ and $\kappa \geq 0$ are Lagrange multipliers. We find the minimum of the Lagrangian with respect to the unconstrained primal variables ω and ξ . Therefore, we have:

$$\omega = \omega_t + \tau_t y_t \mathbf{x}_t,$$

The step-size τ_t will be,

$$\tau_t = \min \left\{ C_t, \frac{\ell^I(\omega_t; (\mathbf{x}_t, y_t))}{\|\mathbf{x}_t\|^2} \right\}. \quad (18)$$

Appendix A.2 details the derivations of the update rules for CSPA₁- ℓ^I .

3.5.3. CSPA₂- ℓ^I

CSPA₂- ℓ^I incorporates the ℓ^I loss function into the CSPA₂ framework, as introduced in Eq. 13. The Lagrangian corresponding to the optimization problem in Eq. 13, formulated using the ℓ^I loss, is given by:

$$\omega_{t+1} = \arg \min_{\omega \in \mathbb{R}^m} \frac{1}{2} \|\omega - \omega_t\|^2 + C_t \xi^2 \quad \text{subject to} \quad \ell^I(\omega; (\mathbf{x}_t, y_t)) \leq \xi, \quad (19)$$

$$\mathcal{L}(\omega, \xi, \tau_t) = \frac{1}{2} \|\omega - \omega_t\|^2 + C_t \xi^2 + \tau_t [\ell^I(\omega; (\mathbf{x}_t, y_t)) - \xi], \quad (20)$$

where $\tau \geq 0$ is Lagrange multipliers (or step size). To determine the optimal solution, we take the derivatives of the Lagrangian with respect to the unconstrained primal variables ω and ξ . Thus, we obtain:

$$\omega = \omega_t + \tau_t y_t \mathbf{x}_t,$$

$$\tau_t = \frac{\ell^I(\omega_t; (\mathbf{x}_t, y_t))}{\|\mathbf{x}_t\|^2 + \frac{1}{2C_t}}. \quad (21)$$

The update rule derivations for CSPA₂- ℓ^I are provided in Appendix A.3.

3.5.4. CSPA- ℓ^{II}

CSPA- ℓ^{II} applies the ℓ^{II} loss function within the standard PA framework, as defined in Eq. 4. The corresponding Lagrangian for the resulting optimization problem is given by:

$$\mathcal{L}(\omega, \tau_t) = \frac{1}{2}\|\omega - \omega_t\|^2 + \tau_t (\ell^{II}(\omega; (\mathbf{x}_t, y_t))), \quad (22)$$

$$\mathcal{L}(\omega, \tau_t) = \frac{1}{2}\|\omega - \omega_t\|^2 + \tau_t [(\rho \cdot \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)}) * \max(0, 1 - y_t(\omega \cdot \mathbf{x}_t))], \quad (23)$$

where $\tau \geq 0$ represents the Lagrange multiplier (or step size). By differentiating the Lagrangian with respect to the variables ω and ξ , and setting the partial derivatives to zero, we derive the closed-form update rules for CSPA- ℓ^{II} algorithm:

$$\frac{\partial \mathcal{L}}{\partial \omega} = \omega - \omega_t - \tau_t(\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)})y_t\mathbf{x}_t = 0, \quad (24)$$

$$\omega = \omega_t + \tau_t(\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)})y_t\mathbf{x}_t, \quad (25)$$

The step-size τ_t will be,

$$\tau_t = \frac{1 - y_t(\omega_t \cdot \mathbf{x}_t)}{(\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)})\|\mathbf{x}_t\|^2}. \quad (26)$$

The derivations of the CSPA- ℓ^{II} update rules are detailed in Appendix B.1.

3.5.5. CSPA₁- ℓ^{II}

CSPA₁- ℓ^{II} extends the CSPA₁ using ℓ^{II} loss function. The Lagrangian associated with the optimization problem in Eq. 11, derived using the ℓ^{II} loss function, is formulated as:

$$\mathcal{L}(\omega, \xi, \tau_t, \lambda) = \frac{1}{2}\|\omega - \omega_t\|^2 + C_t\xi + \tau_t [(\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)}) * \max(0, 1 - y_t(\omega \cdot \mathbf{x}_t)) - \xi] - \kappa\xi, \quad (27)$$

where $\tau \geq 0$ and $\kappa \geq 0$ are Lagrange multipliers. We find the minimum of the Lagrangian with respect to the variables ω and ξ . Therefore, we have:

$$\omega = \omega_t + \tau_t(\rho \cdot I_{(y_t=1)} + I_{(y_t=-1)})y_t\mathbf{x}_t, \quad (28)$$

The step-size τ_t will be,

$$\tau_t = \min \left\{ C_t, \frac{\ell^{II}(\omega; (\mathbf{x}_t, y_t))}{(\rho * I_{(y_t=1)} + I_{(y_t=-1)})^2\|\mathbf{x}_t\|^2} \right\}. \quad (29)$$

The derivations of the CSPA₁- ℓ^{II} update rules are detailed in Appendix B.2.

3.5.6. CSPA₂- ℓ^{II}

CSPA₂- ℓ^{II} incorporates the ℓ^{II} loss function in the CSPA₂ formulation (Eq. 13). The Lagrangian for the optimization problem is given by:

$$\mathcal{L}(\omega, \xi, \tau_t) = \frac{1}{2} \|\omega - \omega_t\|^2 + C_t \xi^2 + \tau_t [(\rho * I_{(y_t=1)} + I_{(y_t=-1)}) * \max(0, 1 - y_t(\omega \cdot \mathbf{x}_t)) - \xi], \quad (30)$$

where $\tau \geq 0$ is a Lagrange multiplier. We find the minimum of the Lagrangian with respect to the variables ω and ξ . Therefore, we have:

$$\omega = \omega_t + \tau_t(\rho * I_{(y_t=1)} + I_{(y_t=-1)})y_t\mathbf{x}_t, \quad (31)$$

The step-size τ_t is given by,

$$\tau_t = \frac{\ell^{II}(\omega_t; (\mathbf{x}_t, y_t))}{(\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)})^2 \|\mathbf{x}_t\|^2 + \frac{1}{2C_t}}. \quad (32)$$

The derivations of the CSPA₂- ℓ^{II} update rules are detailed in Appendix B.3.

Table 1: Variants of the CSPA algorithm and their class imbalance treatment strategies

Algorithm	Where class imbalance treated
CSPA ₁	Cost term C_t in PA ₁
CSPA ₂	Cost term C_t in PA ₂
CSPA- ℓ^I	Loss function ℓ^I in PA
CSPA- ℓ^{II}	Loss function ℓ^{II} in PA
CSPA ₁ - ℓ^I	Both ℓ^I and cost term C_t in PA ₁
CSPA ₁ - ℓ^{II}	Both ℓ^{II} and cost term C_t in PA ₁
CSPA ₂ - ℓ^I	Both ℓ^I and cost term C_t in PA ₂
CSPA ₂ - ℓ^{II}	Both ℓ^{II} and cost term C_t in PA ₂

Algorithm 3 CSPA Algorithms for CCPR

Input: Data $\{(\mathbf{x}_t, y_t)\}_{t=1}^T$; Aggressiveness parameters C , C^+ , and C^- ; Class sizes T_p and T_n ; Loss function type $\ell^*(\omega; (\mathbf{x}_t, y_t))$; Parameter ρ

Output: Final weight vector ω_{T+1}

```

1: Initialize weight vector  $\omega_1 = \mathbf{0}$ 
2: for  $t = 1, \dots, T$  do                                 $\triangleright$  Iterate over each data sample in the stream
3:   Receive instance  $\mathbf{x}_t \in \mathbb{R}^m$  and label  $y_t \in \{-1, +1\}$ 
4:   Predict:  $\hat{y}_t = \text{sign}(\omega_t \cdot \mathbf{x}_t)$ 
5:   Compute loss  $\ell(\omega_t) = \ell^*(\omega_t; (\mathbf{x}_t, y_t))$  where  $*$   $\in \{\text{Hinge}, I, II\}$   $\triangleright \ell_*$  represents Hinge loss,  $\ell^I$ , or  $\ell^{II}$ 
   depending on the algorithm
6:   if  $\ell(\omega_t) > 0$  then                                 $\triangleright$  Update only if loss is positive
7:     Compute penalty  $C_t$  where

```

$$C_t = \begin{cases} C & \text{for PA}_1, \text{PA}_2, \\ C^+ & \text{if } y_t = +1 \text{ for CSPA variants,} \\ C^- & \text{if } y_t = -1 \text{ for CSPA variants} \end{cases}$$

```

8:   Compute step size  $\tau_t^*$  where  $*$   $\in \{\text{PA}, \text{PA}_1, \text{PA}_2\}$ 

```

$$\tau_t^* = \begin{cases} \tau_t^{\text{PA}} & \text{for PA, CSPA-}\ell^I, \text{CSPA-}\ell^I, \\ \tau_t^{\text{PA}_1} & \text{for PA}_1, \text{CSPA}_1, \text{CSPA}_1\text{-}\ell^I, \text{CSPA}_1\text{-}\ell^{II}, \\ \tau_t^{\text{PA}_2} & \text{for PA}_2, \text{CSPA}_2, \text{CSPA}_2\text{-}\ell^I, \text{CSPA}_2\text{-}\ell^{II} \end{cases}$$

```

9:   Update weight vector:

```

$$\omega_{t+1} = \begin{cases} \omega_t + \tau_t^* y_t \mathbf{x}_t & \text{if Hinge or } \ell^I \text{ loss,} \\ \omega_t + \tau_t^* (\rho * \mathbb{I}_{(y_t=1)} + \mathbb{I}_{(y_t=-1)}) y_t \mathbf{x}_t & \text{if } \ell^{II} \text{ loss} \end{cases}$$

```

10:  else
11:     $\omega_{t+1} = \omega_t$                                  $\triangleright$  No update if loss is zero
12:  end if
13: end for
14: return  $\omega_{T+1}$ 

```

3.6. Performance Metrics

We start by defining the set $\mathcal{M} = \{t | y_t \neq \text{sign}(\omega_t \cdot x_t), t = 1, \dots, T\}$, which contains the indices of samples that are misclassified if the model parameter is represented by ω_t . Similarly, we denote by $\mathcal{M}_p = \{t | t \in \mathcal{M} \text{ and } y_t = +1\}$ the set of indices corresponding to false negatives, and $\mathcal{M}_n = \{t | t \in \mathcal{M} \text{ and } y_t = -1\}$ the set of indices corresponding to false positives. We define the following notations: $\mathcal{I}_T^p = \{i \in [T] | y_i = +1\}$, representing the set of indices for positive examples, and $\mathcal{I}_T^n = \{i \in [T] | y_i = -1\}$, representing the set of indices for negative examples. The total number of positive examples is denoted by $T_p = |\mathcal{I}_T^p|$, and the total number of negative examples is denoted by $T_n = |\mathcal{I}_T^n|$. Sensitivity of the classification algorithm, also known as the True Positive Rate (TPR), is defined as:

$$\text{Sensitivity} = \frac{T_p - \mathcal{M}_p}{T_p} \quad (33)$$

Specificity, also known as the True Negative Rate (TNR), quantifies the ability of a model to correctly identify negative samples. It is defined as:

$$\text{Specificity} = \frac{T_n - \mathcal{M}_n}{T_n} \quad (34)$$

G-Mean is the geometric mean of the sensitivity and specificity of the classifier, defined as:

$$\text{G-Mean} = \sqrt{\text{Sensitivity} \times \text{Specificity}} \quad (35)$$

In an online learning framework for binary classification, we assume, without loss of generality, that the positive class is the minority class. This implies that the number of positive samples, T_p , is less than or equal to the number of negative samples, T_n . For simplicity, $\|x_t\| \leq 1$ which is adopted from [32]. In standard online learning, performance is generally assessed using prediction accuracy or, equivalently, the mistake rate over the entire sequence of samples. This approach is not appropriate for imbalanced data, as a machine learning model that classifies all samples as negative could achieve misleadingly high accuracy in cases of severe class imbalance. Therefore, a more effective metric involves evaluating the weighted sum of sensitivity and specificity [32], known as cost-sensitive sum (CSS), which is defined as:

$$\text{CSS} = \eta_p \times \text{Sensitivity} + \eta_n \times \text{Specificity}, \quad (36)$$

where $\eta_p + \eta_n = 1$ and $0 \leq \eta_p, \eta_n \leq 1$ are weights assigned to the positive and negative classes, respectively. When $\eta_p = \eta_n = 0.5$, this metric simplifies to the widely-known balanced accuracy. A higher sum indicates superior classification performance.

The cumulative error rate (CER)/mistake rate at time step t^* is defined as the proportion of misclassified samples observed from the start of the online learning process up to time t^* . It is mathematically defined as:

$$\text{CER}(t^*) = \frac{\sum_{t=1}^{t^*} I(\hat{y}_t \neq y_t)}{t^*}. \quad (37)$$

3.7. Synthetic Data Generation

In addition to a real dataset (which will be discussed in Section 4.3), we will utilize a simulated dataset to study the performance of our proposed algorithms for CCPR. For this purpose, we use a synthetic data generation that produces the abnormal patterns introduced in Figure 1 (b-h). Let T be the number of samples to be received in the entire sequence. Define $\mathbf{x}_t$ as the control chart signal with m features (window length) at an arbitrary time $t = 1, \dots, T$. Further, define μ and σ be the (stationary) mean and standard deviation of a normal process, denoted by R_t . If $r_t \sim \mathcal{N}(0, 1)$, then the signal can be reproduced as

$$\mathbf{x}_t = \mu + r_t \sigma + a_t,$$

where a_t is the abnormal signal at time t , defined as a function specific to the abnormal pattern [30, 53]. The specific equations for generating such patterns are detailed in Table 2. Accordingly, g denotes the gradient magnitude for the trend pattern; for the trend patterns, parameters k and s control the shift position and its magnitude, respectively; the parameter

d indicates the magnitude of the systematic pattern; cyclic patterns are defined using two parameters, Ω and α , which indicate the period of a cycle and the amplitude of the cyclic variation, respectively; and, the term σ' represents the random noise for the stratification pattern.

For each (normal or abnormal) pattern, we generate several control chart pattern datasets by varying the window length m and the parameters controlling the chosen pattern as suggested by Bag et al. [30]; see Table 3. Each dataset then contains data samples of the form $(\mathbf{x}_t, y_t)$ with $y_t \in \{-1, 1\}$, where $+1$ (-1) indicates the input vector $\mathbf{x}_t$ belongs to the abnormal (normal) class.

Table 2: CCPs mathematical models

Pattern	Mathematical Model
Normal	$x_t = \mu + r_t\sigma$
Up-trend	$x_t = \mu + r_t\sigma + gt$
Down-trend	$x_t = \mu + r_t\sigma - gt$
Up-shift	$x_t = \mu + r_t\sigma + ks$
Down-shift	$x_t = \mu + r_t\sigma - ks$
Systematic	$x_t = \mu + r_t\sigma + d(-1)^t$
Cyclic	$x_t = \mu + r_t\sigma + \alpha \sin\left(\frac{2\pi t}{\Omega}\right)$
Stratification	$x_t = \mu + r_t\sigma'$

Table 3: Overview of CCP parameter ranges

Symbol	Pattern	Parameter	Range
m	All	Window length	$\{10, 15, \dots, 100\}$
μ	All	Process mean	0
σ	All	Process standard deviation	1
r_t	All	Random noise	$\mathcal{N}(0, 1)$
g	Trend	Magnitude of gradient	$[0.005\sigma, 0.605\sigma]$
k	Shift	Shift position	$m/2$
s	Shift	Shift magnitude	$[0.005\sigma, 1.805\sigma]$
d	Systematic	Magnitude	$[0.005\sigma, 1.805\sigma]$
α	Cyclic	Amplitude	$[0.005\sigma, 1.805\sigma]$
Ω	Cyclic	Period	8
r'_t	Stratification	Random noise for the pattern	$\mathcal{N}(0, \sigma')$
σ'	Stratification	Standard deviation	$[0.005\sigma, 0.8\sigma]$

4. Computational Results

In this section, we study and compare the performance of standard OGD [25], PA [19], PA_1 [19], and PA_2 [19] algorithms, along with the proposed cost-sensitive algorithms including CSOGD-I and CSOGD-II [32], $CSPA_1$, $CSPA_2$, $CSPA_1-\ell^I$, $CSPA_1-\ell^{II}$, $CSPA_2-\ell^I$, and $CSPA_2-\ell^{II}$ models, on a variety of simulated control chart datasets and a real-world dataset collected from a pharmaceutical manufacturing industry [54]. The (CS)PA and (CS)OGD

algorithms are implemented in Python (version 3.11.3 compiled with GCCcore 12.3.0) and CUDA version 12.3.0, utilizing the LIBOL [48] and NumPy libraries. All experiments are conducted on the University of Oklahoma’s Supercomputing Center for Education and Research (OSCER). The Python code is parallelized to distribute independent simulation tasks, defined by combinations of pattern type, window length, and abnormal parameter, across multiple CPU cores using the concurrent.futures module [55] and the subprocess library [56]. The performance of these algorithms is compared using the evaluation metrics such as accuracy, sensitivity, specificity, precision, mistake count, G-Mean, CSS, computational time, and CER outlined in Section 3.6. For access to our implementations, readers are referred to the online repository listed here.

4.1. Binary Classification with Simulated Data.

The time-series data for both normal and abnormal classes are generated using a simulation model described in Section 3.7. We define imb as the imbalance ratio, computed as the number of abnormal samples divided by the total number of samples:

$$imb = \frac{T_p}{T_p + T_n}$$

where T_p denotes the number of abnormal samples and T_n denotes the number of normal samples. Throughout this section, we use the notation $(1 : \frac{1}{imb})$ to indicate the degree of class imbalance in the dataset. For example, a dataset with an imbalance ratio of $(1 : 10)$ and a total of 1,000 samples would contain $T_p = 100$ abnormal samples and $T_n = 900$ normal samples. For each simulated abnormal pattern type, we consider an imbalanced class distribution of the form $(1 : 10)$, with each dataset consisting of 1,000 samples. Each dataset is generated using a unique combination of window length m and an abnormal parameter value— g , s , a , d , or σ' —corresponding to the specific pattern type. The window length m is varied across 19 distinct values, ranging from 10 to 100 in increments of 5. For each window length and each of the seven pattern types, 40 distinct abnormal parameter values are considered. This results in 760 unique combinations (19×40) and thus 760 distinct datasets per pattern type.

For OGD and CSOGD, a time-decay strategy is applied to the learning rate λ_t , allowing it to decrease with each iteration. Specifically, λ_t is scaled inversely with the square root of the iteration number, which promotes both stability and convergence in the online learning setting [25]. This strategy has been suggested by [48]. Accordingly, the parameter Z is used as a constant to define the time-decaying learning rate, $\lambda_t = \frac{Z}{\sqrt{t}}$, $Z > 0$, where t denotes the iteration number. It enables larger updates during early iterations for rapid learning, while gradually reducing the step size to ensure stable convergence as more data become available. The parameter Z is tuned over the range $\{2^{-4}, 2^{-3}, \dots, 2^7\}$. The weights η_p and η_n are both set to 0.5 to ensure balanced consideration of false positives and false negatives during the training. For CSPA variants, the aggressiveness parameter C is tuned in the range of $\{2^{-4}, 2^{-3}, \dots, 2^7\}$, as suggested in [19, 32]. To optimize the hyperparameters Z and C , we employ the Optuna library [57], which leverages Bayesian optimization to efficiently search the parameter space. The training of all algorithms is conducted over 20 iterations, which is consistent with the evaluation procedure used in [48].

4.2. Results and Discussion

We calculated the G-Mean scores for PA [19], PA_1 [19], PA_2 [19], OGD [25], CSOGD-I [32], CSOGD-II [32], $CSPA_1$, $CSPA_2$, $CSPA_1-\ell^I$, $CSPA_1-\ell^{II}$, $CSPA_2-\ell^I$, and $CSPA_2-\ell^{II}$ models across various abnormal pattern datasets in the presence of class imbalance. Figure 2 shows that the OGD-based (OGD, CSOGD-I, CSOGD-II) algorithms have substantially inferior performance compared to the (CS)PA variants for a dataset with up-shift pattern, particularly when the abnormal parameter falls between 0.18 and 1.32. Furthermore, as shown in Figure 2–(a, b), and (c) illustrate that the G-Mean for OGD-based models increases slowly as the abnormality parameter increases, revealing that these models struggle to detect subtle or early-stage process faults. As a result, alerts are delayed until the deviations become more severe. This trend remains consistent across various window lengths, including both short and long intervals. This limitation is largely mitigated by the PA-based algorithms, which achieve better classification performance and more stability. However, the standard PA, PA_1 , and PA_2 models demonstrate their own practical challenge, a significant drop in G-Mean at smaller window lengths, such as $m = 10, 30$, as seen in Figure 2–(d, e, f).

One can observe that the CSPA models outperform the standard PA models although they still struggle with small window lengths, as shown in Figure 2–(g) and (j). Incorporating class-specific penalties, as implemented in $CSPA_1$ and $CSPA_2$, results in notably enhanced G-Mean performance, particularly in problems with smaller window lengths illustrated in Figure 2–(h, i). No significant performance difference is observed among $CSPA_1$, $CSPA_2$, $CSPA_1-\ell^{II}$, and $CSPA_2-\ell^{II}$. However, models such as $CSPA_1-\ell^{II}$ and $CSPA_2-\ell^{II}$ consistently demonstrate higher and more stable G-Mean values, particularly when compared to standard PA, $CSPA-\ell^I$, and $CSPA-\ell^{II}$, particularly at the smallest window size ($m = 10$), as illustrated in Figure 2–(m) and (n).

Previous studies in the literature [8] have demonstrated three distinct categories of offline CCPR problems: (1) fully separable (FS), where the models achieve high G-Mean values, corresponding to the white regions of the figure; (2) partially separable (PS) problems, which lie at the boundary between high and low performance regions (gray regions); and (3) inseparable (IS) problems, where the models consistently fail to distinguish patterns, as indicated by the black regions. Figures 4 and 5 primarily illustrate PS problems within the context of online learning, with difficulty levels ranging from relatively easy cases (light gray) to more challenging ones (dark gray).

Except for the stratification pattern, increasing the window length T and the abnormal pattern parameter generally improves the separability of the problem in most patterns, which results in higher G-Mean values. In contrast, smaller values for these parameters reduce separability, making the classification task more challenging, as reflected by the darker gray regions. This result is consistent with the findings of offline learning CCPR models in the literature [8, 58]. Figure 4–(a) shows a distinct region of poor performance for an up-trend pattern, particularly at small window lengths and lower values of the abnormal parameter. The G-Mean values remain low (darker regions) which indicates difficulty in detecting early and subtle trends. Figure 4–(b, c) reduces the size of the dark region, especially along the bottom-left portion of the heatmap. This improvement is more significant for systematic and cyclic patterns, as shown in Figure 4–(h, i) and Figure 4–(k, l), respectively.

Figure 5 illustrates the performance of PA, CSPA₁, and CSPA₁- ℓ^I algorithms on the stratification pattern. Unlike other abnormal patterns, these figures do not show the typical L-shaped or reverse L-shaped structure that signals varying degrees of separability. Instead, the heatmaps appear uniformly dark and noisy, indicating persistently poor G-Mean values across nearly all combinations of window length and abnormal parameter values. Notably, incorporating the cost-sensitive framework in CSPA₁ and CSPA₁- ℓ^I offers little to no performance improvement over the standard PA algorithm. This shows a limitation in the ability of these models to detect stratification, a pattern characterized by a reduction in process variance rather than a directional shift or trend.

Furthermore, we compared the performance of the standard PA and the CSPA-based models on several representative problems from the PS category. The evaluation metrics reported include sensitivity, specificity, and the combined score CSS. In imbalanced classification settings, metrics such as specificity tend to be dominated by the majority class and therefore do not provide a reliable assessment of overall performance. In contrast, sensitivity better reflects the correct classification of the minority class. To capture a balanced view of performance across both classes, we report the CSS, which is the weighted sum of sensitivity and specificity. Tables 4 and 5 present the average CSS, sensitivity, and specificity (along with standard deviation) for a range of online learning algorithms, including PA, PA₁, PA₂, CSPA, CSPA₁, CSPA₂, and their loss-modified variants using ℓ^I and ℓ^{II} . The results demonstrate that CSPA₁- ℓ^{II} and CSPA₂- ℓ^I consistently outperform other methods, particularly in problems with window sizes $m = 20$ and $m = 30$, and moderate abnormality parameter values in the range $[0.2, 0.7]$ across most abnormal patterns. For instance, in a dataset with a systematic pattern, $m = 20$, and $\sigma = 0.328$, the CSS improves from 54.60% using PA to 62.18% with CSPA₁- ℓ^{II} , demonstrating the superiority of CSPA₁- ℓ^{II} for such problems. This trend is similarly observed in other complex patterns, including up-shift, down-shift, systematic, and cyclic patterns, all of which benefit more from the combined cost-sensitive learning strategies compared to simpler patterns such as up-trend and down-trend.

Moreover, CSPA₁ and CSPA₁- ℓ^{II} demonstrate slight improvements for up-trend and down-trend patterns when the abnormal parameter is small (e.g., $g \leq 0.1$) and $m = 20$. In contrast, CSPA₂- ℓ^I shows better CSS compared to the standard PA algorithms on up-shift, down-shift, systematic, and cyclic patterns, particularly with moderate abnormality levels and smaller window sizes (e.g., $m = 20$). This is significant for industrial monitoring, as it suggests that while standard models might be sufficient for simple process drifts, the CSPA framework is far more capable of reliably detecting complex, abrupt, or periodic faults. However, in cases with very small abnormal parameters (e.g., $\sigma = 0.005$) and short window lengths (e.g., $m = 10$), there is no significant difference between the performance of PA and CSPA variants across all patterns, indicating that the added complexity from combining the class-specific penalties (C^+ and C^-) with modified loss functions (ℓ^I and ℓ^{II}) together in cost-sensitive learning may not yield benefits in such problems. Also, it is worth mentioning that the combined CSPA variants show a more robust behavior (smaller variation over multiple iterations) across most problems.

To assess the variability and robustness of the proposed CSPA methods in comparison with standard PA approaches, we use box plots of the G-Mean metric, as shown in

Figure 6, based on 20 independent runs on the same dataset for each algorithm. As illustrated in the figure, $\text{CSPA}_1\text{-}\ell^{II}$, $\text{CSPA}_2\text{-}\ell^I$, and $\text{CSPA}_2\text{-}\ell^{II}$ are more robust compared to the other algorithms when applied to imbalanced datasets. In particular, $\text{CSPA}_1\text{-}\ell^{II}$ demonstrates the smallest variability, followed by $\text{CSPA}_2\text{-}\ell^I$ and $\text{CSPA}_2\text{-}\ell^{II}$, especially for shift, systematic, and cyclic patterns. In contrast, no substantial differences in variability are observed among the algorithms for the up-trend and down-trend patterns.

Next, we studied the impact of varying window lengths and abnormal parameter intensities on the performance of the PA and CSPA algorithms in terms of G-Mean. First, we selected a representative problem from each abnormal pattern category and applied each algorithm using datasets with varying window lengths $m \in \{10, 25, 50, 75, 95\}$. The average G-Mean values are reported in Table 6 and Table 7. As shown in these tables, $\text{CSPA}_1\text{-}\ell^{II}$ consistently outperforms all competing algorithms, achieving the highest performance in 22 out of the evaluated cases. However, for up-trend and down-trend patterns with a low abnormality value ($g = 0.015$), $\text{CSPA}_2\text{-}\ell^I$ performs significantly better at a short window length ($m = 10$) and slightly better results at $m = 25$. This is a critical finding for production systems where detecting a fault early, within a small data window, can prevent significant material loss or equipment damage. For larger window lengths ($m = 50, 75, 95$), the differences in G-Mean performance between PA[19] and CSPA variants in up- and down-trend patterns become negligible. However, for more complex patterns such as up-shift, down-shift, and systematic, the CSPA variants consistently outperform the PA algorithms across all window lengths. For example, in the up-shift pattern with $m = 25$, the best-performing CSPA model, $\text{CSPA}_2\text{-}\ell^{II}$, achieves a G-Mean of 0.573, substantially higher than the 0.526 achieved by the standard PA model. For the cyclic pattern, $\text{CSPA}_1\text{-}\ell^{II}$ yields the best results across most window lengths, with particularly superior performance at the smallest and largest window sizes (i.e., $m = 10$ and $m = 95$).

Second, Table 7 presents the G-Mean values for varying abnormality parameters (e.g., g, s, d, α) while keeping the window length fixed at $m = 20$. We observe that for up-/down-trend patterns with low to moderate abnormality parameters ($g = 0.051$ and $g = 0.097$), the CSPA algorithms consistently outperform the PA models. However, the performance difference becomes minimal at the extremes, when the abnormality parameter is very small ($g = 0.005$) or very large ($g = 0.220, 0.590$). For up-/down-shift patterns, CSPA algorithms demonstrate superior G-Mean performance at larger abnormal pattern parameters ($s = 0.236, 0.467, 0.559$). Similar trends are observed for systematic and cyclic patterns, where CSPA consistently outperforms PA, particularly under high abnormality intensities.

In our experiments, the difference in computational time between PA and CSPA algorithms was negligible. In particular, we report the computational time for PA, CSPA_1 , and $\text{CSPA}_1\text{-}\ell^I$ in Figure 7. One could observe that the computational time increases approximately linearly with the number of samples, which is expected for online learning methods. On a dataset with a cyclic pattern and 1,000 data instances, the CSPA_1 and $\text{CSPA}_1\text{-}\ell^I$ algorithms perform the entire online learning in 0.25 seconds or less on a standard computing machine. The experiment was performed with window lengths of 10, 30, 70, and 100.

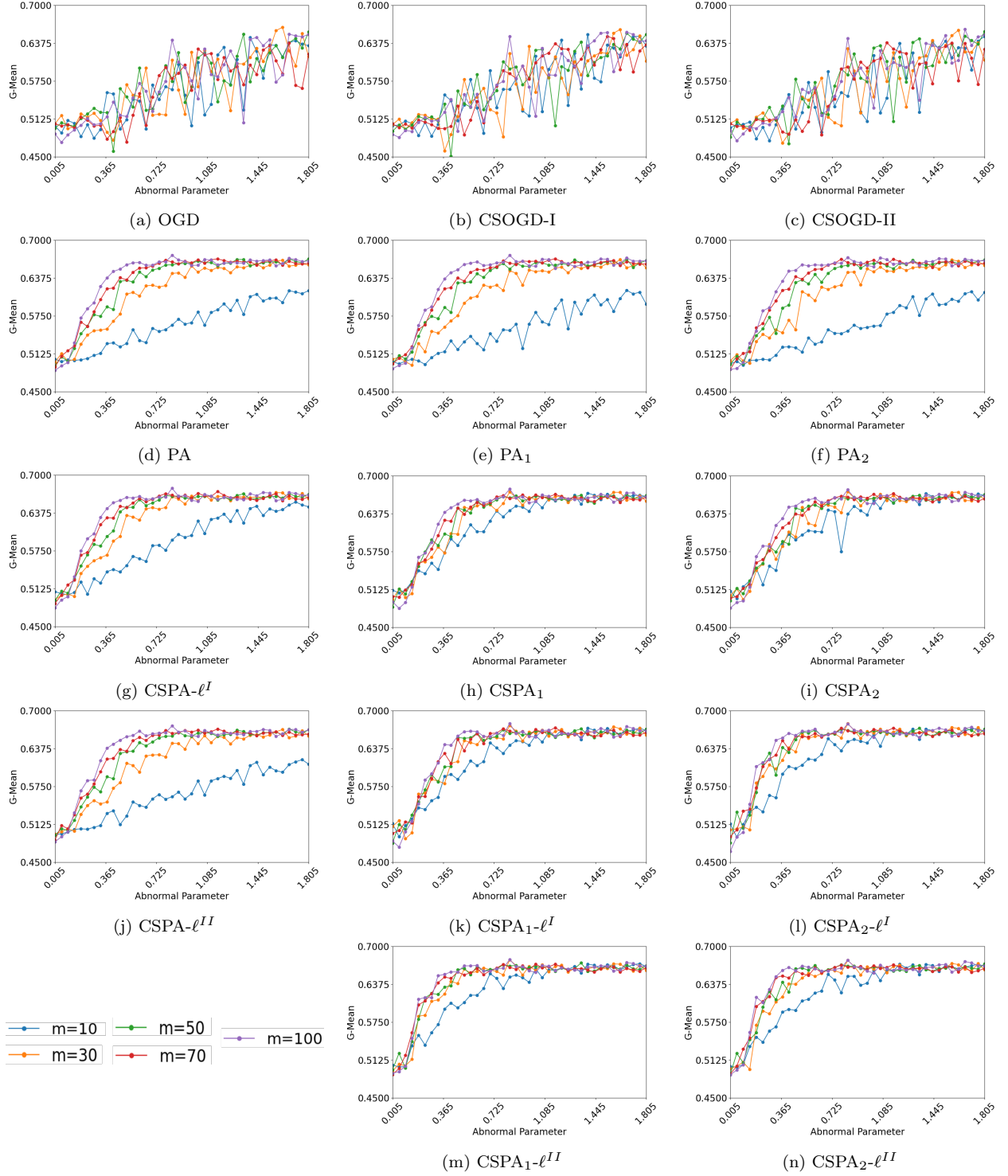


Figure 2: Comparison among the PA, PA_1 , PA_2 [19]; OGD [25]; CSOGD-I and CSOGD-II [32]; and the proposed $CSPA_1$, $CSPA_2$, $CSPA_1-\ell^I$, $CSPA_1-\ell^{II}$, $CSPA_2-\ell^I$, and $CSPA_2-\ell^{II}$ algorithms for the up-shift pattern with $m \in \{10, 30, 50, 70, 100\}$ and $s \in [0.005, 1.805]$.

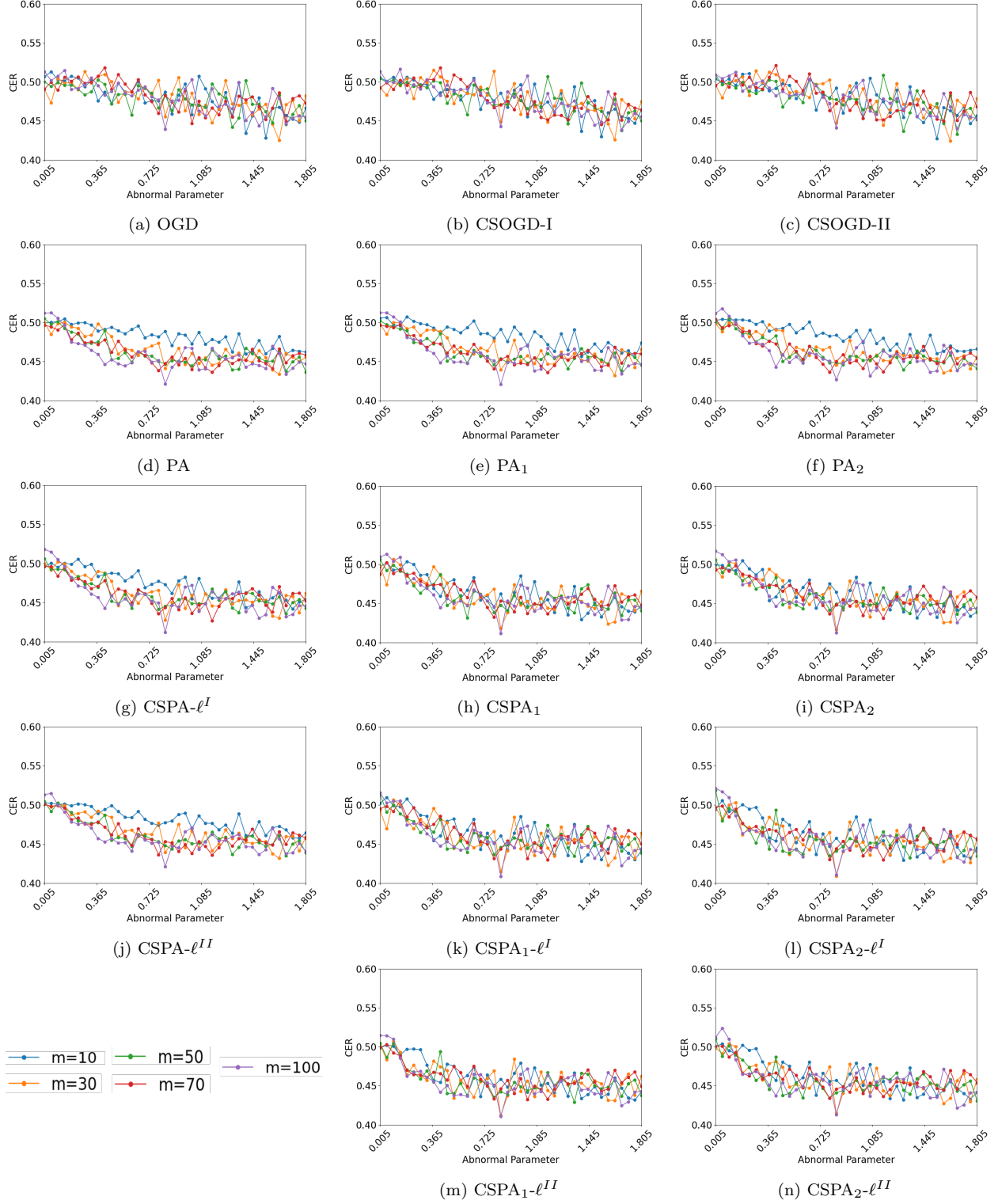


Figure 3: CER Comparison among the PA, PA₁, PA₂ [19], OGD [25], CSOGD-I, CSOGD-II [32], CSPA₁, CSPA₂, CSPA₁- ℓ^I , CSPA₁- ℓ^{II} , CSPA₂- ℓ^I , and CSPA₂- ℓ^{II} algorithms for a up-shift pattern with $m \in \{10, 30, 50, 70, 100\}$ and $s \in [0.005, 1.805]$.

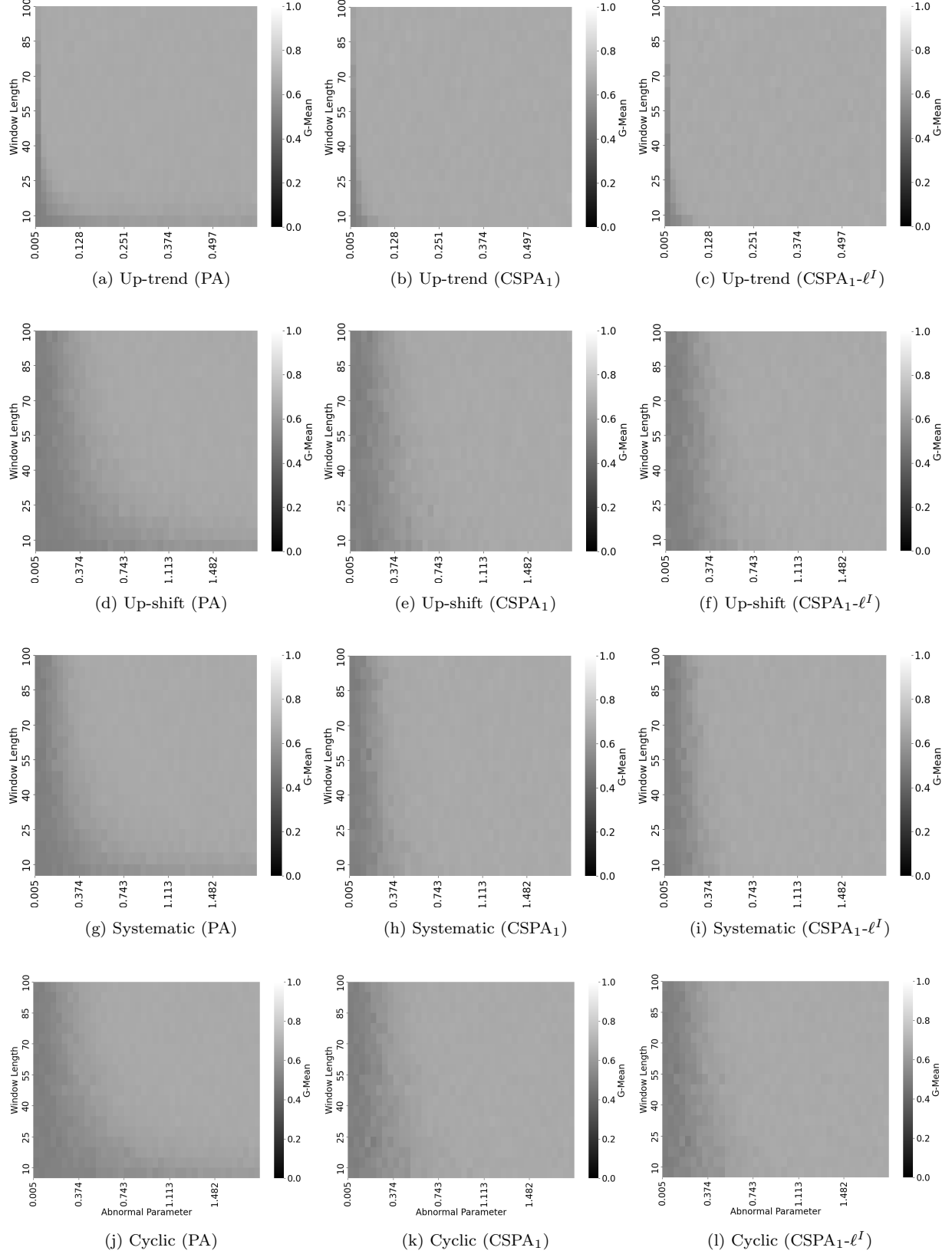


Figure 4: G-Mean performance of PA [19], CSPA₁, and CSPA₁- ℓ^I algorithms across different abnormal pattern types

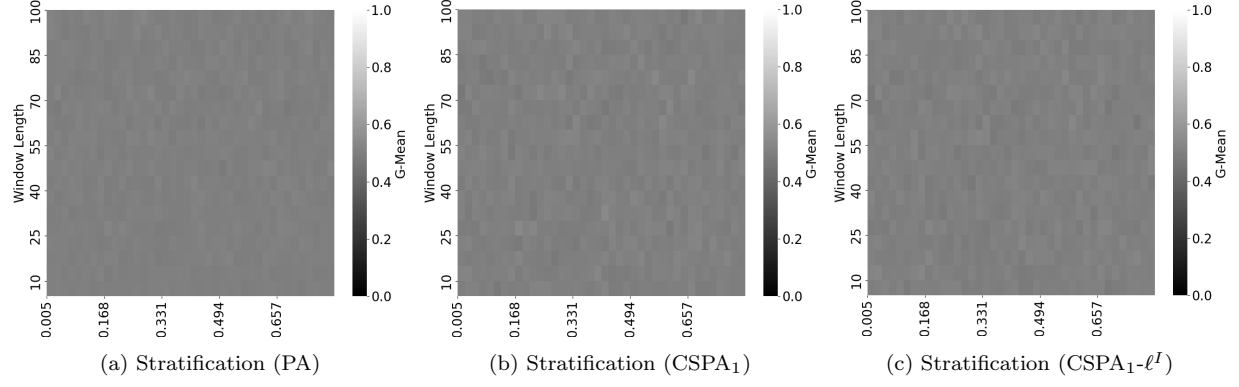


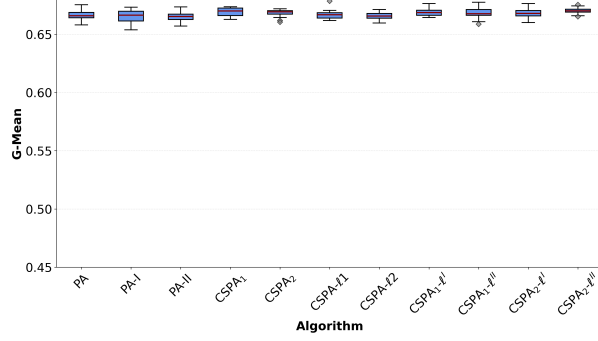
Figure 5: G-Mean performance of PA [19], CSPA₁, and CSPA₁- ℓ^I algorithms for the stratification pattern

Table 4: Average CSS, Sensitivity, and Specificity (with standard deviation) of the PA, PA₁, PA₂ [19], CSPA₁, CSPA₂, CSPA_{1-ℓ^I}, CSPA_{1-ℓ^{II}}, CSPA_{2-ℓ^I}, and CSPA_{2-ℓ^{II}} algorithms for the up-trend, down-trend, and up-shift patterns. The highest sensitivity, specificity, and CSS is denoted with bold font.

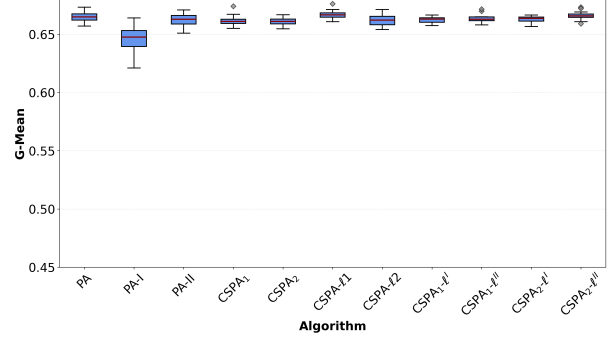
Algorithm	Up-trend, $m = 25, g = 0.128$			Up-trend, $m = 20, g = 0.097$			Up-trend, $m = 10, g = 0.005$		
	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)
PA [19]	66.120 ± 0.428	73.647 ± 0.643	58.593 ± 0.215	64.070 ± 0.977	70.611 ± 1.446	57.529 ± 0.510	50.166 ± 1.705	50.244 ± 2.506	50.088 ± 0.904
PA ₁ [19]	66.061 ± 0.503	73.547 ± 0.760	58.576 ± 0.247	64.055 ± 1.184	70.606 ± 1.748	57.505 ± 0.619	49.775 ± 1.345	49.669 ± 1.978	49.881 ± 0.712
PA ₂ [19]	66.208 ± 0.559	73.761 ± 0.832	58.654 ± 0.288	64.068 ± 0.632	70.608 ± 0.930	57.527 ± 0.337	50.070 ± 1.367	50.103 ± 2.009	50.037 ± 0.724
CSPA ₁	66.982 ± 0.440	74.906 ± 0.672	59.058 ± 0.208	66.013 ± 0.525	73.411 ± 0.807	58.614 ± 0.245	49.548 ± 1.467	49.336 ± 2.157	49.761 ± 0.777
CSPA ₂	66.997 ± 0.325	74.939 ± 0.490	59.055 ± 0.160	65.712 ± 0.348	72.942 ± 0.525	58.483 ± 0.173	49.828 ± 1.642	49.747 ± 2.414	49.909 ± 0.870
CSPA _{1-ℓ^I}	66.945 ± 0.548	74.858 ± 0.838	59.031 ± 0.258	66.006 ± 0.566	73.442 ± 0.863	58.570 ± 0.271	50.160 ± 1.758	50.236 ± 2.586	50.085 ± 0.931
CSPA _{1-ℓ^{II}}	66.335 ± 0.531	73.956 ± 0.806	58.715 ± 0.257	64.331 ± 0.895	70.986 ± 1.334	57.677 ± 0.457	48.772 ± 1.679	48.194 ± 2.469	49.349 ± 0.890
CSPA _{1-ℓ^I}	66.943 ± 0.230	74.856 ± 0.346	59.031 ± 0.115	65.746 ± 0.303	73.000 ± 0.460	58.492 ± 0.150	49.713 ± 1.605	49.578 ± 2.360	49.848 ± 0.850
CSPA _{1-ℓ^{II}}	66.655 ± 0.415	74.417 ± 0.636	58.893 ± 0.196	65.880 ± 0.322	73.203 ± 0.505	58.558 ± 0.142	49.873 ± 1.156	49.814 ± 1.700	49.933 ± 0.612
CSPA _{2-ℓ^I}	66.971 ± 0.498	74.886 ± 0.764	59.056 ± 0.233	65.640 ± 0.226	72.819 ± 0.339	58.461 ± 0.113	49.728 ± 2.182	49.600 ± 3.208	49.856 ± 1.156
CSPA _{2-ℓ^{II}}	66.881 ± 0.363	74.758 ± 0.553	59.003 ± 0.174	65.905 ± 0.432	73.278 ± 0.645	58.532 ± 0.220	50.569 ± 1.077	50.836 ± 1.583	50.301 ± 0.570
Down-trend, $m = 25, g = 0.128$									
Algorithm	Down-trend, $m = 25, g = 0.128$			Down-trend, $m = 20, g = 0.097$			Down-trend, $m = 10, g = 0.005$		
	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)
PA [19]	66.893 ± 0.673	74.794 ± 1.012	58.991 ± 0.334	64.821 ± 0.835	71.736 ± 1.244	57.905 ± 0.426	50.597 ± 2.044	50.878 ± 3.006	50.316 ± 1.083
PA ₁ [19]	66.441 ± 0.470	74.114 ± 0.710	58.768 ± 0.230	64.690 ± 0.986	71.533 ± 1.463	57.847 ± 0.511	50.287 ± 1.897	50.422 ± 2.789	50.152 ± 1.004
PA ₂ [19]	66.548 ± 0.612	74.275 ± 0.928	58.821 ± 0.297	65.018 ± 0.688	72.019 ± 1.033	58.016 ± 0.346	49.614 ± 1.460	49.433 ± 2.147	49.795 ± 0.774
CSPA ₁	67.017 ± 0.421	74.961 ± 0.648	59.072 ± 0.196	66.550 ± 0.586	74.258 ± 0.891	58.842 ± 0.281	50.174 ± 1.732	50.256 ± 2.547	50.092 ± 0.917
CSPA ₂	66.925 ± 0.373	74.842 ± 0.563	59.009 ± 0.184	66.319 ± 0.667	73.911 ± 1.015	58.727 ± 0.320	49.939 ± 1.439	49.911 ± 2.116	49.968 ± 0.763
CSPA _{1-ℓ^I}	67.058 ± 0.493	75.031 ± 0.760	59.085 ± 0.225	66.348 ± 0.327	73.947 ± 0.498	58.750 ± 0.159	50.340 ± 1.722	50.500 ± 2.532	50.180 ± 0.912
CSPA _{1-ℓ^{II}}	66.493 ± 0.585	74.186 ± 0.879	58.800 ± 0.291	64.978 ± 0.686	71.972 ± 1.020	57.984 ± 0.353	50.794 ± 1.296	51.167 ± 1.905	50.420 ± 0.686
CSPA _{1-ℓ^I}	66.977 ± 0.310	74.906 ± 0.465	59.049 ± 0.154	66.517 ± 0.322	74.203 ± 0.485	58.831 ± 0.161	49.783 ± 1.366	49.681 ± 2.008	49.884 ± 0.724
CSPA _{1-ℓ^{II}}	67.183 ± 0.229	75.219 ± 0.343	59.146 ± 0.116	66.598 ± 0.441	74.325 ± 0.671	58.871 ± 0.211	49.465 ± 1.061	49.214 ± 1.560	49.717 ± 0.562
CSPA _{2-ℓ^I}	67.095 ± 0.226	75.086 ± 0.338	59.104 ± 0.114	66.302 ± 0.252	73.869 ± 0.389	58.735 ± 0.117	50.359 ± 1.390	50.528 ± 2.044	50.191 ± 0.737
CSPA _{2-ℓ^{II}}	66.890 ± 0.245	74.778 ± 0.373	59.001 ± 0.117	66.544 ± 0.522	74.250 ± 0.789	58.839 ± 0.255	48.865 ± 1.668	48.331 ± 2.452	49.399 ± 0.883
Up-shift, $m = 35, s = 0.743$									
Algorithm	Up-shift, $m = 35, s = 0.743$			Up-shift, $m = 20, s = 0.236$			Up-shift, $m = 10, s = 0.005$		
	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)
PA [19]	64.833 ± 0.727	71.775 ± 1.080	57.892 ± 0.374	52.420 ± 1.268	53.558 ± 1.865	51.282 ± 0.672	50.329 ± 1.494	50.483 ± 2.196	50.174 ± 0.792
PA ₁ [19]	64.150 ± 0.901	70.761 ± 1.335	57.540 ± 0.468	49.885 ± 1.750	49.831 ± 2.572	49.939 ± 0.927	49.932 ± 1.589	49.900 ± 2.337	49.964 ± 0.842
PA ₂ [19]	63.935 ± 1.095	70.442 ± 1.619	57.428 ± 0.573	52.619 ± 1.842	53.850 ± 2.708	51.388 ± 0.976	49.556 ± 1.498	49.347 ± 2.203	49.765 ± 0.793
CSPA ₁	65.798 ± 0.752	73.172 ± 1.117	58.423 ± 0.387	55.115 ± 1.474	57.519 ± 2.167	52.711 ± 0.781	51.101 ± 1.159	51.619 ± 1.705	50.583 ± 0.614
CSPA ₂	65.662 ± 0.687	72.981 ± 1.018	58.344 ± 0.357	55.943 ± 1.022	58.739 ± 1.502	53.148 ± 0.541	50.941 ± 1.770	51.383 ± 2.603	50.498 ± 0.938
CSPA _{1-ℓ^I}	65.642 ± 0.685	72.947 ± 1.007	58.337 ± 0.364	53.171 ± 1.686	54.661 ± 2.480	51.680 ± 0.893	50.737 ± 1.963	51.083 ± 2.886	50.390 ± 1.040
CSPA _{1-ℓ^{II}}	64.094 ± 0.780	70.683 ± 1.143	57.505 ± 0.418	52.683 ± 1.297	53.944 ± 1.907	51.421 ± 0.687	49.507 ± 2.129	49.275 ± 3.129	49.739 ± 1.128
CSPA _{1-ℓ^I}	66.553 ± 0.365	74.281 ± 0.544	58.825 ± 0.187	56.556 ± 1.123	59.639 ± 1.651	53.474 ± 0.596	51.362 ± 1.542	52.003 ± 2.267	50.722 ± 0.817
CSPA _{1-ℓ^{II}}	66.779 ± 0.305	74.614 ± 0.454	58.944 ± 0.157	58.057 ± 0.881	61.842 ± 1.294	54.272 ± 0.469	50.406 ± 1.373	50.597 ± 2.018	50.215 ± 0.727
CSPA _{2-ℓ^I}	66.708 ± 0.404	74.506 ± 0.617	58.910 ± 0.193	54.777 ± 1.195	57.022 ± 1.756	52.531 ± 0.633	51.287 ± 1.174	51.892 ± 1.727	50.682 ± 0.622
CSPA _{2-ℓ^{II}}	66.909 ± 0.334	74.811 ± 0.500	59.007 ± 0.169	57.857 ± 0.977	61.550 ± 1.437	54.164 ± 0.517	50.164 ± 1.643	50.242 ± 2.416	50.087 ± 0.870

Table 5: Average CSS, Sensitivity, and Specificity (with standard deviation) of the PA, PA₁, PA₂ [19], CSPA₁, CSPA₂, CSPA₁- ℓ^I , CSPA₁- ℓ^{II} , CSPA₂- ℓ^I , and CSPA₂- ℓ^{II} algorithms for the down-shift, systematic, and cyclic patterns. The highest sensitivity, specificity, and CSS is denoted with bold font.

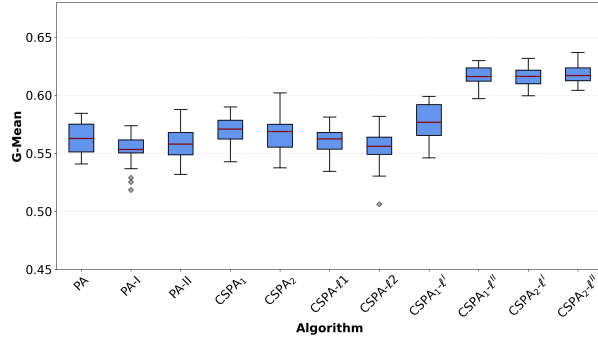
Algorithm	Down-shift, $m = 35, s = 0.743$			Down-shift, $m = 20, s = 0.236$			Down-shift, $m = 10, s = 0.005$		
	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)
PA [19]	64.512 ± 1.258	71.297 ± 1.864	57.727 ± 0.652	52.067 ± 1.485	53.039 ± 2.184	51.095 ± 0.786	49.803 ± 1.584	49.711 ± 2.329	49.896 ± 0.839
PA ₁ [19]	63.944 ± 1.049	70.481 ± 1.547	57.408 ± 0.552	52.154 ± 1.782	53.167 ± 2.619	51.142 ± 0.944	49.917 ± 1.722	49.878 ± 2.532	49.956 ± 0.912
PA ₂ [19]	64.021 ± 1.119	70.586 ± 1.651	57.456 ± 0.588	52.092 ± 1.772	53.075 ± 2.605	51.109 ± 0.939	50.015 ± 1.311	50.022 ± 1.928	50.008 ± 0.695
CSPA ₁	66.772 ± 0.518	74.625 ± 0.768	58.918 ± 0.269	55.360 ± 1.328	57.878 ± 1.952	52.842 ± 0.704	49.247 ± 1.076	48.894 ± 1.580	49.600 ± 0.571
CSPA ₂	66.288 ± 0.682	73.911 ± 1.020	58.666 ± 0.346	56.012 ± 1.411	58.833 ± 2.070	53.190 ± 0.752	50.429 ± 1.653	50.631 ± 2.431	50.227 ± 0.876
CSPA ₁ - ℓ^I	65.772 ± 0.646	73.142 ± 0.963	58.402 ± 0.331	53.105 ± 1.286	54.564 ± 1.891	51.646 ± 0.681	49.666 ± 1.559	49.508 ± 2.292	49.823 ± 0.826
CSPA ₁ - ℓ^{II}	64.372 ± 0.808	71.094 ± 1.182	57.650 ± 0.434	51.374 ± 1.134	52.019 ± 1.666	50.728 ± 0.601	50.146 ± 1.400	50.214 ± 2.059	50.078 ± 0.742
CSPA ₂ - ℓ^I	66.928 ± 0.429	74.853 ± 0.638	59.003 ± 0.220	56.224 ± 1.209	59.144 ± 1.777	53.303 ± 0.640	49.395 ± 1.630	49.111 ± 2.396	49.680 ± 0.864
CSPA ₂ - ℓ^{II}	67.152 ± 0.257	75.178 ± 0.386	59.126 ± 0.128	58.463 ± 0.699	62.425 ± 1.031	54.500 ± 0.367	49.356 ± 1.590	49.053 ± 2.337	49.658 ± 0.842
CSPA ₂ - ℓ^I	67.076 ± 0.285	75.061 ± 0.425	59.091 ± 0.145	58.848 ± 0.862	62.989 ± 1.263	54.706 ± 0.461	47.448 ± 0.981	46.247 ± 1.443	48.648 ± 0.520
CSPA ₂ - ℓ^{II}	66.807 ± 0.523	74.664 ± 0.788	58.950 ± 0.259	55.893 ± 1.362	58.656 ± 2.002	53.130 ± 0.722	50.263 ± 1.663	50.386 ± 2.445	50.139 ± 0.881
Systematic, $m = 40, \sigma = 0.513$									
	Down-shift, $m = 20, s = 0.282$			Down-shift, $m = 10, s = 0.005$			Cyclic, $m = 15, s = 0.005$		
	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)
PA [19]	65.258 ± 0.886	72.408 ± 1.302	58.107 ± 0.469	54.604 ± 1.527	56.767 ± 2.244	52.442 ± 0.811	49.622 ± 1.755	49.444 ± 2.579	49.800 ± 0.930
PA ₁ [19]	64.225 ± 1.094	70.886 ± 1.614	57.564 ± 0.574	51.124 ± 1.603	51.653 ± 2.356	50.596 ± 0.849	50.017 ± 1.084	50.025 ± 1.594	50.009 ± 0.575
PA ₂ [19]	64.898 ± 0.750	71.869 ± 1.108	57.927 ± 0.393	54.533 ± 1.433	56.664 ± 2.107	52.402 ± 0.760	49.680 ± 2.331	49.531 ± 3.427	49.830 ± 1.235
CSPA ₁	66.487 ± 0.649	74.192 ± 0.973	58.782 ± 0.327	60.845 ± 1.078	65.919 ± 1.581	55.770 ± 0.575	49.258 ± 1.271	48.908 ± 1.869	49.607 ± 0.673
CSPA ₂	65.663 ± 0.768	72.980 ± 1.130	58.338 ± 0.406	59.865 ± 1.135	64.489 ± 1.659	55.242 ± 0.611	48.595 ± 1.394	47.933 ± 2.049	49.256 ± 0.738
CSPA ₁ - ℓ^I	66.501 ± 0.551	74.217 ± 0.819	58.785 ± 0.283	56.465 ± 1.069	59.500 ± 1.573	53.431 ± 0.566	49.836 ± 1.939	49.758 ± 2.851	49.913 ± 1.027
CSPA ₁ - ℓ^{II}	65.004 ± 0.778	72.031 ± 1.150	57.978 ± 0.406	54.653 ± 1.919	56.839 ± 2.822	52.466 ± 1.017	49.033 ± 1.337	48.578 ± 1.966	49.488 ± 0.709
CSPA ₂ - ℓ^I	66.930 ± 0.521	74.850 ± 0.770	59.010 ± 0.272	60.448 ± 0.867	65.339 ± 1.276	55.556 ± 0.459	49.297 ± 1.974	48.967 ± 2.902	49.628 ± 1.045
CSPA ₂ - ℓ^{II}	66.950 ± 0.404	74.875 ± 0.605	59.025 ± 0.205	62.181 ± 0.757	67.853 ± 1.113	56.509 ± 0.402	48.492 ± 1.441	47.783 ± 2.119	49.202 ± 0.763
CSPA ₂ - ℓ^I	66.821 ± 0.443	74.689 ± 0.671	58.954 ± 0.216	61.988 ± 0.881	67.586 ± 1.298	56.390 ± 0.465	48.772 ± 1.435	48.194 ± 2.110	49.350 ± 0.760
CSPA ₂ - ℓ^{II}	67.187 ± 0.359	75.231 ± 0.541	59.144 ± 0.177	61.405 ± 0.980	66.731 ± 1.438	56.080 ± 0.522	48.570 ± 0.948	47.897 ± 1.393	49.242 ± 0.502
Cyclic, $m = 30, s = 0.697$									
	Down-shift, $m = 20, s = 0.282$			Down-shift, $m = 10, s = 0.005$			Cyclic, $m = 15, s = 0.005$		
	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)	CSS (%)	Sensitivity (%)	Specificity (%)
PA [19]	62.613 ± 1.147	68.528 ± 1.686	56.699 ± 0.608	52.402 ± 1.572	53.531 ± 2.311	51.273 ± 0.833	49.188 ± 1.572	48.806 ± 2.311	49.570 ± 0.832
PA ₁ [19]	62.050 ± 0.720	67.697 ± 1.064	56.403 ± 0.376	53.114 ± 1.984	54.578 ± 2.916	51.651 ± 1.051	49.957 ± 1.105	49.936 ± 1.624	49.977 ± 0.586
PA ₂ [19]	62.725 ± 1.116	68.686 ± 1.644	56.763 ± 0.590	51.466 ± 1.642	52.156 ± 2.415	50.777 ± 0.870	49.739 ± 1.882	49.617 ± 2.768	49.861 ± 0.997
CSPA ₁	64.796 ± 0.804	71.708 ± 1.177	57.885 ± 0.432	57.314 ± 1.230	60.750 ± 1.806	53.879 ± 0.653	49.904 ± 1.462	49.858 ± 2.149	49.949 ± 0.775
CSPA ₂	66.389 ± 0.470	74.064 ± 0.698	58.714 ± 0.242	56.807 ± 1.256	60.003 ± 1.844	53.610 ± 0.667	49.826 ± 1.710	49.744 ± 2.514	49.908 ± 0.906
CSPA ₁ - ℓ^I	64.732 ± 1.144	71.628 ± 1.689	57.837 ± 0.600	53.516 ± 1.169	55.167 ± 1.718	51.865 ± 0.619	50.089 ± 1.977	50.131 ± 2.906	50.047 ± 1.048
CSPA ₁ - ℓ^{II}	62.844 ± 1.111	68.858 ± 1.637	56.829 ± 0.585	53.237 ± 1.298	54.758 ± 1.908	51.715 ± 0.687	50.580 ± 2.222	50.853 ± 3.268	50.307 ± 1.177
CSPA ₂ - ℓ^I	66.425 ± 0.566	74.108 ± 0.828	58.743 ± 0.305	55.774 ± 1.397	58.489 ± 2.054	53.060 ± 0.739	48.834 ± 1.388	48.286 ± 2.040	49.382 ± 0.736
CSPA ₂ - ℓ^{II}	67.019 ± 0.311	74.978 ± 0.464	59.060 ± 0.159	58.440 ± 1.123	62.406 ± 1.653	54.474 ± 0.593	48.387 ± 1.206	47.628 ± 1.774	49.146 ± 0.639
CSPA ₂ - ℓ^I	67.057 ± 0.368	75.036 ± 0.553	59.078 ± 0.184	59.226 ± 1.044	63.561 ± 1.534	54.892 ± 0.554	49.460 ± 1.329	49.206 ± 1.954	49.714 ± 0.705
CSPA ₂ - ℓ^{II}	66.359 ± 0.511	74.008 ± 0.763	58.710 ± 0.259	58.887 ± 0.935	63.061 ± 1.375	54.713 ± 0.495	49.734 ± 1.604	49.608 ± 2.358	49.859 ± 0.849



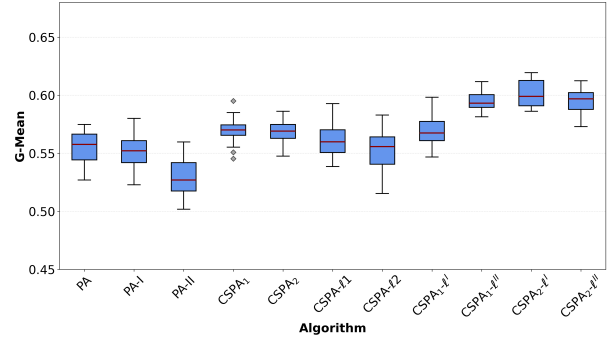
(a) Up-trend pattern ($g = 0.051$)



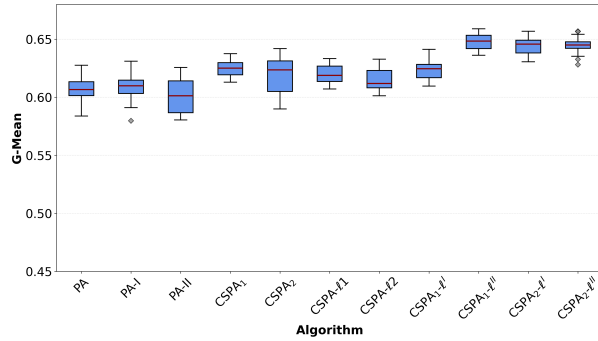
(b) Down-trend pattern ($g = 0.051$)



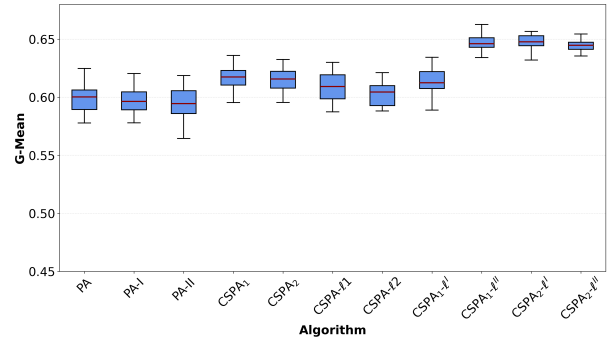
(c) Up-shift pattern ($s = 0.236$)



(d) Down-shift pattern ($s = 0.236$)



(e) Systematic pattern ($d = 0.282$)



(f) Cyclic pattern ($a = 0.42$)

Figure 6: Robustness analysis of the proposed CSPA variants in comparison with standard PA approaches. The box plots show the distribution of G-Mean values across abnormal patterns with window length $m = 50$, aggregated over 20 iterations.

Table 6: Comparative results of the proposed CSPA methods and the PA baseline in terms of G-Mean across varying window lengths and abnormal patterns. The best-performing results are highlighted in bold, and the count of wins is determined based on the highest G-Mean value. The last column reports the G-Mean improvement of the best-performing CSPA relative to the best PA baseline method. Improvement values exceeding 5% are highlighted in bold.

Pattern	m	Parameters	PA [10]	PA ₁ [10]	PA ₂ [19]	CSPA ₁	CSPA ₂	CSPA- ℓ^I	CSPA- ℓ^{II}	CSPA ₁ - ℓ^I	CSPA ₁ - ℓ^{II}	CSPA ₂ - ℓ^I	CSPA ₂ - ℓ^{II}	G-Mean Improv.
UT	10		0.528 ± 0.016	0.530 ± 0.024	0.528 ± 0.015	0.603 ± 0.009	0.604 ± 0.009	0.550 ± 0.017	0.534 ± 0.016	0.595 ± 0.011	0.598 ± 0.010	0.611 ± 0.006	0.609 ± 0.008	15%
	25		0.642 ± 0.006	0.640 ± 0.009	0.640 ± 0.008	0.655 ± 0.004	0.646 ± 0.007	0.655 ± 0.005	0.638 ± 0.007	0.652 ± 0.004	0.656 ± 0.002	0.657 ± 0.004	0.654 ± 0.005	2.6%
	50	$g = 0.051$	0.666 ± 0.004	0.664 ± 0.004	0.664 ± 0.008	0.669 ± 0.003	0.669 ± 0.003	0.667 ± 0.003	0.666 ± 0.005	0.669 ± 0.003	0.668 ± 0.004	0.668 ± 0.002	0.667 ± 0.004	0.45%
	75		0.662 ± 0.005	0.664 ± 0.004	0.663 ± 0.004	0.663 ± 0.004	0.664 ± 0.004	0.662 ± 0.004	0.663 ± 0.004	0.662 ± 0.002	0.662 ± 0.003	0.659 ± 0.002	0.659 ± 0.002	0%
	95		0.662 ± 0.004	0.663 ± 0.006	0.663 ± 0.004	0.665 ± 0.003	0.662 ± 0.004	0.663 ± 0.003	0.662 ± 0.004	0.662 ± 0.004	0.665 ± 0.003	0.665 ± 0.002	0.663 ± 0.002	0.3%
DT	10		0.519 ± 0.010	0.519 ± 0.014	0.511 ± 0.017	0.570 ± 0.012	0.569 ± 0.008	0.530 ± 0.016	0.516 ± 0.015	0.562 ± 0.013	0.576 ± 0.007	0.581 ± 0.010	0.579 ± 0.009	11.95%
	25		0.636 ± 0.009	0.634 ± 0.012	0.635 ± 0.012	0.656 ± 0.005	0.657 ± 0.004	0.656 ± 0.006	0.636 ± 0.009	0.657 ± 0.005	0.660 ± 0.002	0.660 ± 0.003	0.659 ± 0.003	3.77%
	50	$g = 0.051$	0.663 ± 0.005	0.665 ± 0.006	0.664 ± 0.004	0.662 ± 0.003	0.665 ± 0.005	0.663 ± 0.004	0.663 ± 0.004	0.661 ± 0.004	0.663 ± 0.004	0.664 ± 0.004	0.665 ± 0.005	0%
	75		0.663 ± 0.003	0.664 ± 0.004	0.663 ± 0.004	0.662 ± 0.005	0.664 ± 0.003	0.663 ± 0.004	0.664 ± 0.005	0.666 ± 0.003	0.663 ± 0.004	0.667 ± 0.002	0.666 ± 0.002	0.44%
	95		0.670 ± 0.005	0.669 ± 0.004	0.669 ± 0.004	0.670 ± 0.005	0.668 ± 0.003	0.671 ± 0.004	0.669 ± 0.004	0.669 ± 0.003	0.668 ± 0.005	0.669 ± 0.003	0.672 ± 0.003	0.30%
US	10		0.504 ± 0.018	0.495 ± 0.020	0.503 ± 0.017	0.539 ± 0.013	0.520 ± 0.019	0.503 ± 0.017	0.504 ± 0.020	0.537 ± 0.012	0.537 ± 0.014	0.533 ± 0.013	0.542 ± 0.008	7.5%
	25		0.526 ± 0.020	0.522 ± 0.013	0.518 ± 0.019	0.538 ± 0.012	0.543 ± 0.016	0.533 ± 0.017	0.521 ± 0.013	0.546 ± 0.012	0.571 ± 0.007	0.570 ± 0.009	0.573 ± 0.008	8.2%
	50	$s = 0.236$	0.558 ± 0.015	0.564 ± 0.019	0.554 ± 0.013	0.574 ± 0.012	0.555 ± 0.016	0.568 ± 0.013	0.558 ± 0.017	0.569 ± 0.012	0.615 ± 0.008	0.612 ± 0.010	0.600 ± 0.010	8.3%
	75		0.578 ± 0.012	0.581 ± 0.011	0.573 ± 0.011	0.572 ± 0.015	0.573 ± 0.013	0.585 ± 0.012	0.579 ± 0.016	0.569 ± 0.012	0.620 ± 0.008	0.625 ± 0.007	0.628 ± 0.010	7.5%
	95		0.598 ± 0.010	0.591 ± 0.011	0.591 ± 0.012	0.600 ± 0.009	0.596 ± 0.013	0.600 ± 0.012	0.590 ± 0.014	0.603 ± 0.013	0.628 ± 0.010	0.629 ± 0.010	0.630 ± 0.009	5.1%
DS	10		0.511 ± 0.011	0.501 ± 0.014	0.507 ± 0.021	0.543 ± 0.010	0.527 ± 0.016	0.509 ± 0.020	0.517 ± 0.019	0.533 ± 0.014	0.544 ± 0.010	0.532 ± 0.014	0.548 ± 0.009	7.2%
	25		0.538 ± 0.015	0.535 ± 0.012	0.534 ± 0.020	0.561 ± 0.014	0.574 ± 0.012	0.549 ± 0.018	0.536 ± 0.013	0.573 ± 0.014	0.594 ± 0.007	0.562 ± 0.011	0.560 ± 0.013	9.4%
	50	$s = 0.236$	0.552 ± 0.013	0.560 ± 0.012	0.541 ± 0.015	0.564 ± 0.013	0.557 ± 0.014	0.560 ± 0.016	0.555 ± 0.010	0.573 ± 0.012	0.599 ± 0.008	0.600 ± 0.011	0.595 ± 0.008	6.7%
	75		0.586 ± 0.014	0.570 ± 0.010	0.583 ± 0.011	0.584 ± 0.015	0.576 ± 0.011	0.598 ± 0.010	0.582 ± 0.014	0.594 ± 0.011	0.611 ± 0.009	0.618 ± 0.009	0.629 ± 0.010	7.3%
	95		0.601 ± 0.010	0.604 ± 0.012	0.602 ± 0.011	0.596 ± 0.016	0.601 ± 0.011	0.610 ± 0.011	0.604 ± 0.011	0.611 ± 0.011	0.636 ± 0.007	0.633 ± 0.009	0.631 ± 0.006	5.3%
SYS	10		0.528 ± 0.015	0.525 ± 0.013	0.523 ± 0.011	0.584 ± 0.008	0.592 ± 0.010	0.538 ± 0.021	0.529 ± 0.012	0.589 ± 0.010	0.590 ± 0.007	0.587 ± 0.011	0.589 ± 0.008	12.1%
	25		0.541 ± 0.016	0.517 ± 0.019	0.534 ± 0.014	0.569 ± 0.012	0.573 ± 0.010	0.557 ± 0.014	0.540 ± 0.012	0.576 ± 0.013	0.587 ± 0.008	0.592 ± 0.009	0.591 ± 0.009	9.4%
	50	$d = 0.236$	0.585 ± 0.012	0.587 ± 0.014	0.585 ± 0.011	0.601 ± 0.012	0.596 ± 0.012	0.603 ± 0.010	0.590 ± 0.013	0.598 ± 0.013	0.638 ± 0.007	0.628 ± 0.008	0.638 ± 0.009	9.1%
	75		0.599 ± 0.013	0.600 ± 0.011	0.599 ± 0.011	0.599 ± 0.013	0.598 ± 0.012	0.612 ± 0.012	0.602 ± 0.011	0.614 ± 0.013	0.640 ± 0.007	0.642 ± 0.010	0.640 ± 0.005	7%
	95		0.617 ± 0.011	0.592 ± 0.015	0.614 ± 0.011	0.619 ± 0.010	0.612 ± 0.011	0.624 ± 0.009	0.616 ± 0.012	0.595 ± 0.010	0.639 ± 0.009	0.636 ± 0.008	0.647 ± 0.005	4.9%
CYC	10		0.511 ± 0.018	0.507 ± 0.021	0.503 ± 0.019	0.528 ± 0.014	0.510 ± 0.016	0.517 ± 0.016	0.506 ± 0.019	0.535 ± 0.009	0.539 ± 0.011	0.534 ± 0.014	0.535 ± 0.007	5.5%
	25		0.501 ± 0.013	0.504 ± 0.016	0.505 ± 0.021	0.500 ± 0.014	0.508 ± 0.014	0.500 ± 0.018	0.500 ± 0.015	0.501 ± 0.016	0.509 ± 0.022	0.503 ± 0.011	0.497 ± 0.016	0.79%
	50	$a = 0.097$	0.507 ± 0.017	0.495 ± 0.018	0.503 ± 0.017	0.504 ± 0.013	0.499 ± 0.017	0.505 ± 0.012	0.499 ± 0.016	0.494 ± 0.012	0.517 ± 0.012	0.507 ± 0.014	0.512 ± 0.012	1.97%
	75		0.516 ± 0.020	0.502 ± 0.018	0.510 ± 0.012	0.503 ± 0.022	0.500 ± 0.015	0.515 ± 0.015	0.510 ± 0.017	0.497 ± 0.016	0.531 ± 0.010	0.536 ± 0.010	0.519 ± 0.018	3.88%
	95		0.519 ± 0.014	0.521 ± 0.015	0.516 ± 0.013	0.520 ± 0.012	0.522 ± 0.016	0.517 ± 0.014	0.524 ± 0.015	0.517 ± 0.014	0.545 ± 0.013	0.530 ± 0.012	0.545 ± 0.009	5%
Count of wins			0	1	0	2	4	0	0	1	9	10	10	

Table 7: Comparative results of the proposed CSPA methods and the PA baseline in terms of G-Mean across varying abnormal patterns and abnormal parameters for each pattern. The best-performing results are highlighted in bold, and the count of wins is determined based on the highest G-Mean value. The last column reports the G-Mean improvement of the best-performing CSPA relative to the best PA baseline method. Improvement values exceeding 5% are highlighted in bold.

Pattern	Parameters	m	PA [49]	PA ₁ [49]	PA ₂ [49]	CSPA _{A1}	CSPA _{A2}	CSPA- ℓ^I	CSPA- ℓ^{II}	CSPA _{A1} - ℓ^I	CSPA _{A1} - ℓ^{II}	CSPA _{A2} - ℓ^I	CSPA _{A2} - ℓ^{II}	G-Mean Improv.
UT	$g = 0.005$		0.507 $\pm$ 0.020	0.504 $\pm$ 0.017	0.504 $\pm$ 0.018	0.507 $\pm$ 0.011	0.502 $\pm$ 0.015	0.506 $\pm$ 0.013	0.503 $\pm$ 0.014	0.513 $\pm$ 0.016	0.513 $\pm$ 0.011	0.511 $\pm$ 0.013	0.507 $\pm$ 0.016	1.18%
	$g = 0.051$		0.607 $\pm$ 0.009	0.601 $\pm$ 0.017	0.607 $\pm$ 0.011	0.651 $\pm$ 0.007	0.652 $\pm$ 0.007	0.634 $\pm$ 0.008	0.611 $\pm$ 0.013	0.655 $\pm$ 0.005	0.656 $\pm$ 0.005	0.658 $\pm$ 0.004	0.658 $\pm$ 0.004	8.40%
	$g = 0.097$	20	0.637 $\pm$ 0.009	0.637 $\pm$ 0.011	0.637 $\pm$ 0.006	0.656 $\pm$ 0.005	0.653 $\pm$ 0.003	0.656 $\pm$ 0.005	0.640 $\pm$ 0.009	0.653 $\pm$ 0.003	0.655 $\pm$ 0.003	0.652 $\pm$ 0.002	0.655 $\pm$ 0.004	2.98%
	$g = 0.220$		0.652 $\pm$ 0.007	0.649 $\pm$ 0.009	0.649 $\pm$ 0.008	0.660 $\pm$ 0.003	0.660 $\pm$ 0.004	0.660 $\pm$ 0.005	0.652 $\pm$ 0.007	0.658 $\pm$ 0.003	0.659 $\pm$ 0.003	0.660 $\pm$ 0.002	0.658 $\pm$ 0.002	1.23%
	$g = 0.590$		0.655 $\pm$ 0.009	0.632 $\pm$ 0.021	0.656 $\pm$ 0.006	0.667 $\pm$ 0.004	0.666 $\pm$ 0.004	0.663 $\pm$ 0.005	0.654 $\pm$ 0.006	0.664 $\pm$ 0.003	0.665 $\pm$ 0.005	0.664 $\pm$ 0.003	0.664 $\pm$ 0.004	1.68%
DT	$g = 0.005$		0.497 $\pm$ 0.017	0.498 $\pm$ 0.018	0.501 $\pm$ 0.017	0.499 $\pm$ 0.021	0.498 $\pm$ 0.016	0.497 $\pm$ 0.013	0.498 $\pm$ 0.016	0.494 $\pm$ 0.015	0.505 $\pm$ 0.013	0.504 $\pm$ 0.018	0.495 $\pm$ 0.021	0.80%
	$g = 0.051$		0.621 $\pm$ 0.010	0.623 $\pm$ 0.011	0.621 $\pm$ 0.017	0.659 $\pm$ 0.008	0.665 $\pm$ 0.005	0.652 $\pm$ 0.006	0.623 $\pm$ 0.009	0.666 $\pm$ 0.006	0.672 $\pm$ 0.004	0.667 $\pm$ 0.004	0.669 $\pm$ 0.004	7.87%
	$g = 0.097$	20	0.644 $\pm$ 0.008	0.643 $\pm$ 0.009	0.646 $\pm$ 0.007	0.661 $\pm$ 0.006	0.659 $\pm$ 0.006	0.659 $\pm$ 0.003	0.646 $\pm$ 0.007	0.661 $\pm$ 0.003	0.661 $\pm$ 0.004	0.659 $\pm$ 0.002	0.661 $\pm$ 0.005	2.32%
	$g = 0.220$		0.653 $\pm$ 0.007	0.656 $\pm$ 0.007	0.656 $\pm$ 0.006	0.662 $\pm$ 0.003	0.661 $\pm$ 0.005	0.663 $\pm$ 0.004	0.655 $\pm$ 0.006	0.661 $\pm$ 0.003	0.662 $\pm$ 0.006	0.662 $\pm$ 0.002	0.661 $\pm$ 0.004	1.07%
	$g = 0.590$		0.659 $\pm$ 0.007	0.662 $\pm$ 0.005	0.659 $\pm$ 0.007	0.671 $\pm$ 0.003	0.667 $\pm$ 0.004	0.665 $\pm$ 0.006	0.662 $\pm$ 0.004	0.667 $\pm$ 0.003	0.671 $\pm$ 0.003	0.667 $\pm$ 0.002	0.667 $\pm$ 0.002	1.36%
US	$s = 0.005$		0.503 $\pm$ 0.018	0.500 $\pm$ 0.012	0.500 $\pm$ 0.015	0.505 $\pm$ 0.016	0.501 $\pm$ 0.017	0.507 $\pm$ 0.011	0.507 $\pm$ 0.015	0.501 $\pm$ 0.016	0.513 $\pm$ 0.011	0.507 $\pm$ 0.012	0.517 $\pm$ 0.015	2.78%
	$s = 0.097$		0.491 $\pm$ 0.019	0.496 $\pm$ 0.020	0.499 $\pm$ 0.019	0.488 $\pm$ 0.014	0.491 $\pm$ 0.015	0.498 $\pm$ 0.018	0.501 $\pm$ 0.014	0.482 $\pm$ 0.015	0.480 $\pm$ 0.013	0.495 $\pm$ 0.016	0.493 $\pm$ 0.011	0.40%
	$s = 0.236$	20	0.524 $\pm$ 0.013	0.499 $\pm$ 0.018	0.526 $\pm$ 0.018	0.551 $\pm$ 0.014	0.559 $\pm$ 0.010	0.531 $\pm$ 0.017	0.527 $\pm$ 0.013	0.565 $\pm$ 0.011	0.579 $\pm$ 0.009	0.547 $\pm$ 0.012	0.577 $\pm$ 0.010	10.08%
	$s = 0.467$		0.576 $\pm$ 0.017	0.577 $\pm$ 0.011	0.576 $\pm$ 0.014	0.612 $\pm$ 0.013	0.638 $\pm$ 0.009	0.592 $\pm$ 0.014	0.575 $\pm$ 0.021	0.635 $\pm$ 0.008	0.646 $\pm$ 0.009	0.643 $\pm$ 0.006	0.641 $\pm$ 0.006	11.96%
	$s = 0.559$		0.574 $\pm$ 0.018	0.578 $\pm$ 0.012	0.574 $\pm$ 0.013	0.639 $\pm$ 0.009	0.647 $\pm$ 0.007	0.605 $\pm$ 0.011	0.580 $\pm$ 0.012	0.639 $\pm$ 0.009	0.638 $\pm$ 0.009	0.643 $\pm$ 0.008	0.652 $\pm$ 0.005	12.80%
DS	$s = 0.005$		0.501 $\pm$ 0.015	0.492 $\pm$ 0.016	0.502 $\pm$ 0.015	0.496 $\pm$ 0.017	0.496 $\pm$ 0.017	0.498 $\pm$ 0.022	0.495 $\pm$ 0.018	0.498 $\pm$ 0.017	0.492 $\pm$ 0.017	0.486 $\pm$ 0.013	0.485 $\pm$ 0.013	-0.80%
	$s = 0.097$		0.505 $\pm$ 0.021	0.503 $\pm$ 0.015	0.507 $\pm$ 0.016	0.510 $\pm$ 0.012	0.509 $\pm$ 0.013	0.504 $\pm$ 0.018	0.495 $\pm$ 0.022	0.512 $\pm$ 0.015	0.506 $\pm$ 0.015	0.500 $\pm$ 0.020	0.503 $\pm$ 0.016	0.99%
	$s = 0.236$	20	0.521 $\pm$ 0.015	0.521 $\pm$ 0.018	0.521 $\pm$ 0.018	0.553 $\pm$ 0.013	0.559 $\pm$ 0.014	0.531 $\pm$ 0.013	0.514 $\pm$ 0.011	0.561 $\pm$ 0.012	0.583 $\pm$ 0.007	0.587 $\pm$ 0.008	0.558 $\pm$ 0.013	12.67%
	$s = 0.467$		0.545 $\pm$ 0.016	0.547 $\pm$ 0.010	0.537 $\pm$ 0.017	0.580 $\pm$ 0.016	0.589 $\pm$ 0.012	0.562 $\pm$ 0.013	0.548 $\pm$ 0.015	0.593 $\pm$ 0.012	0.613 $\pm$ 0.009	0.612 $\pm$ 0.010	0.613 $\pm$ 0.009	12.07%
	$s = 0.559$		0.574 $\pm$ 0.011	0.570 $\pm$ 0.012	0.569 $\pm$ 0.013	0.629 $\pm$ 0.008	0.633 $\pm$ 0.008	0.599 $\pm$ 0.013	0.574 $\pm$ 0.013	0.631 $\pm$ 0.010	0.635 $\pm$ 0.006	0.641 $\pm$ 0.005	0.618 $\pm$ 0.008	11.67%
SYS	$d = 0.005$		0.498 $\pm$ 0.021	0.498 $\pm$ 0.019	0.504 $\pm$ 0.021	0.503 $\pm$ 0.013	0.498 $\pm$ 0.013	0.496 $\pm$ 0.019	0.499 $\pm$ 0.017	0.499 $\pm$ 0.014	0.510 $\pm$ 0.011	0.505 $\pm$ 0.017	0.506 $\pm$ 0.017	1.19%
	$d = 0.051$		0.495 $\pm$ 0.019	0.496 $\pm$ 0.016	0.503 $\pm$ 0.016	0.505 $\pm$ 0.014	0.498 $\pm$ 0.020	0.507 $\pm$ 0.018	0.498 $\pm$ 0.014	0.503 $\pm$ 0.017	0.505 $\pm$ 0.017	0.503 $\pm$ 0.012	0.500 $\pm$ 0.010	0.80%
	$d = 0.097$	20	0.511 $\pm$ 0.015	0.509 $\pm$ 0.013	0.508 $\pm$ 0.017	0.514 $\pm$ 0.017	0.515 $\pm$ 0.017	0.521 $\pm$ 0.021	0.515 $\pm$ 0.015	0.522 $\pm$ 0.011	0.542 $\pm$ 0.013	0.542 $\pm$ 0.011	0.541 $\pm$ 0.010	6.06%
	$d = 0.236$		0.523 $\pm$ 0.014	0.527 $\pm$ 0.016	0.506 $\pm$ 0.020	0.545 $\pm$ 0.016	0.551 $\pm$ 0.011	0.537 $\pm$ 0.016	0.530 $\pm$ 0.016	0.555 $\pm$ 0.016	0.569 $\pm$ 0.012	0.590 $\pm$ 0.009	0.587 $\pm$ 0.010	11.95%
	$d = 0.559$		0.626 $\pm$ 0.013	0.583 $\pm$ 0.015	0.625 $\pm$ 0.012	0.660 $\pm$ 0.006	0.669 $\pm$ 0.004	0.648 $\pm$ 0.009	0.622 $\pm$ 0.013	0.670 $\pm$ 0.004	0.671 $\pm$ 0.005	0.670 $\pm$ 0.004	0.668 $\pm$ 0.005	7.19%
CYC	$a = 0.005$		0.505 $\pm$ 0.020	0.504 $\pm$ 0.017	0.502 $\pm$ 0.015	0.500 $\pm$ 0.017	0.495 $\pm$ 0.015	0.503 $\pm$ 0.020	0.502 $\pm$ 0.017	0.493 $\pm$ 0.015	0.506 $\pm$ 0.017	0.493 $\pm$ 0.014	0.500 $\pm$ 0.009	0.39%
	$a = 0.051$		0.510 $\pm$ 0.012	0.513 $\pm$ 0.018	0.512 $\pm$ 0.013	0.509 $\pm$ 0.019	0.520 $\pm$ 0.016	0.505 $\pm$ 0.017	0.511 $\pm$ 0.020	0.525 $\pm$ 0.010	0.515 $\pm$ 0.013	0.524 $\pm$ 0.012	0.506 $\pm$ 0.014	2.34%
	$a = 0.097$	20	0.500 $\pm$ 0.020	0.504 $\pm$ 0.017	0.509 $\pm$ 0.020	0.525 $\pm$ 0.012	0.507 $\pm$ 0.014	0.516 $\pm$ 0.013	0.507 $\pm$ 0.014	0.520 $\pm$ 0.012	0.516 $\pm$ 0.015	0.526 $\pm$ 0.013	0.523 $\pm$ 0.012	3.34%
	$a = 0.236$		0.512 $\pm$ 0.019	0.509 $\pm$ 0.018	0.505 $\pm$ 0.017	0.544 $\pm$ 0.011	0.536 $\pm$ 0.010	0.528 $\pm$ 0.013	0.519 $\pm$ 0.016	0.532 $\pm$ 0.019	0.578 $\pm$ 0.012	0.575 $\pm$ 0.011	0.574 $\pm$ 0.012	12.89%
	$a = 0.559$		0.581 $\pm$ 0.010	0.574 $\pm$ 0.013	0.580 $\pm$ 0.018	0.626 $\pm$ 0.010	0.630 $\pm$ 0.008	0.598 $\pm$ 0.014	0.576 $\pm$ 0.010	0.630 $\pm$ 0.008	0.642 $\pm$ 0.005	0.643 $\pm$ 0.005	0.641 $\pm$ 0.008	10.67%
Count of wins			0	0	1	5	1	4	1	4	13	8	5	

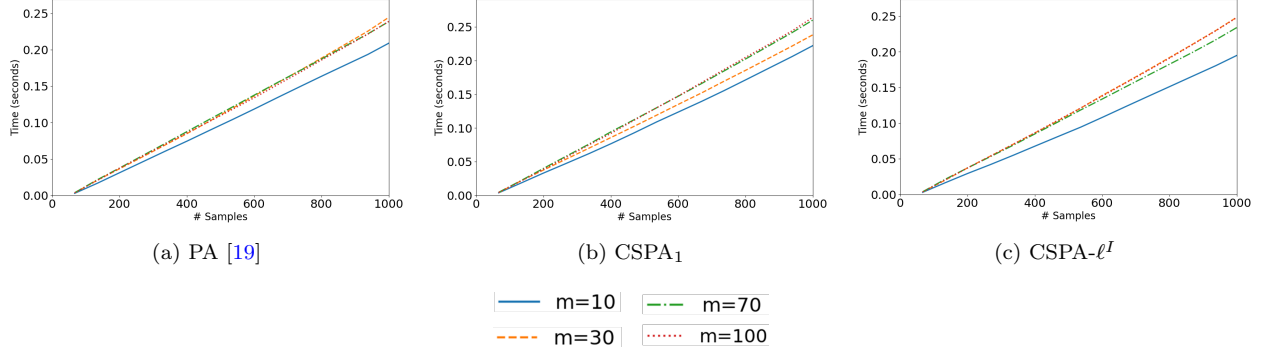


Figure 7: Computational time for PA [19], CSPA₁, and CSPA- ℓ^I algorithms for a cyclic pattern with $m \in \{10, 30, 70, 100\}$ and $a = 0.559$.

To statistically assess performance differences among the algorithms, we first apply the Friedman chi-squared test [59] to evaluate whether the G-Mean varies significantly across models under six abnormal patterns (up-trend, down-trend, up-shift, down-shift, systematic, and cyclic), using a fixed window length of $m = 20$. The null hypothesis (H_0) assumes equivalent performance across all algorithms, whereas the alternative hypothesis (H_1) states that at least one algorithm performs significantly different. The Friedman test strongly rejected H_0 , yielding $\chi_F^2 = 51.12$, an F -statistic of 28.79, and a p -value < 0.001 , which indicates statistically significant performance differences among the eleven evaluated algorithms. We subsequently apply the Nemenyi post-hoc test [60] to conduct pairwise statistical comparisons between models. Average ranks obtained from the Friedman test are reported, and statistically significant differences are identified using the Nemenyi test [61]. The resulting rankings and pairwise comparison outcomes are summarized in Tables 8 and 9. As reported in Table 8, the proposed CSPA variants (CSPA₂- ℓ^I , CSPA₁- ℓ^{II} , and CSPA₂- ℓ^{II}) obtain the lowest average ranks (2.33–2.50), while the standard PA algorithms (PA, PA-I, and PA-II) yield markedly higher average ranks (9.17–10.33). In addition, CSPA₂- ℓ^I demonstrates a statistically significant advantage over CSPA- ℓ^{II} , with a rank difference of 6.17.

Table 8: Average algorithm rankings from the Nemenyi post-hoc test computed over six abnormal patterns (window length $m = 20$ with varying abnormal parameters). Lower ranks indicate superior G-Mean performance within each pattern.

Pattern	Param.	PA	PA-I	PA-II	CSPA ₁	CSPA ₂	CSPA- ℓ^I	CSPA- ℓ^{II}	CSPA ₁ - ℓ^I	CSPA ₁ - ℓ^{II}	CSPA ₂ - ℓ^I	CSPA ₂ - ℓ^{II}
UT	$g = 0.051$	10	11	9	6	5	7	8	4	3	2	1
DT	$g = 0.051$	11	9	10	6	5	7	8	4	1	3	2
US	$s = 0.236$	10	11	9	5	4	7	8	3	1	6	2
DS	$s = 0.236$	10	8	9	6	4	7	11	3	2	1	5
SYS	$d = 0.236$	10	9	11	6	5	7	8	4	3	1	2
CYC	$a = 0.097$	11	10	7	2	9	6	8	4	5	1	3
Average Rank		10.33	9.67	9.17	5.17	5.33	6.83	8.50	3.67	2.50	2.33	2.50

The pairwise p -values obtained from the Nemenyi test are reported in Table 9. The Nemenyi test indicates statistically significant performance differences among classifiers whose average-rank differences exceed the critical difference ($CD = 6.16$; p -value < 0.05). Specifically, Table 9 shows that CSPA₁- ℓ^{II} , CSPA₂- ℓ^I , and CSPA₂- ℓ^{II} demonstrate statistically

significant improvements over PA, PA-I, and PA-II. In contrast, no statistically significant differences are observed among the top-performing CSPA variants themselves, indicating comparable performance within this group. These findings provide strong statistical evidence that the proposed CSPA framework, incorporating modified loss functions, yields significant and robust performance gains for detecting abnormal patterns in imbalanced online CCPR problems. For brevity and due to page limitations, we report results for $m = 20$; however, the same statistical testing framework can be simply applied to other window lengths and problem settings.

Table 9: Pairwise comparison p-values obtained from the Nemenyi post-hoc test for the CCPR problems, as reported in Table 8. Bold values indicate statistical significance differences ($p < 0.05$).

	PA	PA-I	PA-II	CSPA ₁	CSPA ₂	CSPA- ℓ^I	CSPA- ℓ^{II}	CSPA ₁ - ℓ^I	CSPA ₁ - ℓ^{II}	CSPA ₂ - ℓ^I	CSPA ₂ - ℓ^{II}
PA	–	1.000	1.000	0.200	0.243	0.764	0.997	0.021	0.002	0.001	0.002
PA-I	1.000	–	1.000	0.399	0.460	0.927	1.000	0.065	0.008	0.006	0.008
PA-II	1.000	1.000	–	0.585	0.648	0.981	1.000	0.132	0.021	0.016	0.021
CSPA₁	0.200	0.399	0.585	–	1.000	0.999	0.814	0.999	0.951	0.927	0.951
CSPA₂	0.243	0.460	0.648	1.000	–	0.999	0.859	0.999	0.927	0.896	0.927
CSPA-ℓ^I	0.764	0.927	0.981	0.999	0.999	–	0.999	0.859	0.460	0.399	0.460
CSPA-ℓ^{II}	0.997	1.000	1.000	0.814	0.859	0.999	–	0.290	0.065	0.050	0.065
CSPA₁-ℓ^I	0.021	0.065	0.132	0.999	0.999	0.859	0.290	–	1.000	1.000	1.000
CSPA₁-ℓ^{II}	0.002	0.008	0.021	0.951	0.927	0.460	0.065	1.000	–	1.000	1.000
CSPA₂-ℓ^I	0.001	0.006	0.016	0.927	0.896	0.399	0.050	1.000	1.000	–	1.000
CSPA₂-ℓ^{II}	0.002	0.008	0.021	0.951	0.927	0.460	0.065	1.000	1.000	1.000	–

To further evaluate the proposed models, we compare the CSPA algorithms with several existing cost-sensitive online learning algorithms, including CSRDA-I and CSRDA-II [35], as well as CSTG-I and CSTG-II [34], using multiple imbalanced datasets characterized by a range of abnormal patterns (see Table 10). For CSRDA [35], the regularization parameters λ and γ are tuned over the ranges $\{10^{-4}, 10^{-3.5}, \dots, 10^0\}$ and $\{10^{-5}, 10^{-4.5}, \dots, 10^{-1}\}$, respectively, with the sliding window length fixed at 100. For CSTG [34], the penalty parameter C is tuned over $\{2^{-4}, 2^{-3}, \dots, 2^7\}$, the learning rate η over $\{10^{-3}, 10^{-2.5}, \dots, 10^{0.5}\}$, the sparsity penalty g over $\{10^{-2}, 10^{-1.5}, \dots, 10^{1.5}\}$, and the truncation interval K over $\{5, 10, 15, \dots, 50\}$, following the experimental setup in [34]. To ensure a fair comparison, the misclassification cost weights for both CSRDA and CSTG are set to $\eta_p = 0.5$ and $\eta_n = 0.5$, consistent with the configuration used for the CSPA algorithms. As shown in Table 10, the proposed CSPA methods substantially outperform the existing online cost-sensitive learning algorithms across all abnormal patterns, achieving higher G-Mean values compared to CSRDA-I, CSRDA-II, CSTG-I, and CSTG-II. The performance superiority is most substantial for shift and systematic patterns, where the best-performing variants, CSPA₁- ℓ^{II} and CSPA₂- ℓ^I , yield G-Mean improvements of up to 27% compare to the existing online cost-sensitive learning methods.

Table 10: Comparison of G-Mean performance between the proposed CSPA methods and existing state-of-the-art online cost-sensitive learning techniques including CSRDA-I and CSRDA-II [35], as well as CSTG-I and CSTG-II [34].

Pattern	m	Param.	Our Algorithms								From Literature			
			CSPA ₁	CSPA ₂	CSPA- ℓ^I	CSPA- ℓ^{II}	CSPA ₁ - ℓ^I	CSPA ₁ - ℓ^{II}	CSPA ₂ - ℓ^I	CSPA ₂ - ℓ^{II}	CSRDA-I [35]	CSRDA-II [35]	CSTG-I [34]	CSTG-II [34]
UT	10		0.587	0.593	0.536	0.522	0.588	0.597	0.589	0.586	0.502	0.529	0.512	0.510
	25		0.664	0.665	0.661	0.646	0.664	0.664	0.667	0.665	0.548	0.613	0.612	0.625
	50	$g = 0.051$	0.665	0.665	0.669	0.665	0.670	0.673	0.673	0.672	0.623	0.648	0.612	0.641
	75		0.661	0.661	0.660	0.659	0.663	0.665	0.663	0.664	0.591	0.627	0.632	0.630
	95		0.664	0.665	0.663	0.665	0.665	0.666	0.665	0.664	0.628	0.648	0.652	0.645
DT	10		0.587	0.587	0.537	0.525	0.582	0.594	0.593	0.589	0.506	0.526	0.519	0.506
	25		0.658	0.660	0.659	0.642	0.661	0.660	0.660	0.659	0.499	0.614	0.583	0.589
	50	$g = 0.051$	0.670	0.670	0.667	0.667	0.670	0.670	0.672	0.669	0.607	0.650	0.657	0.660
	75		0.666	0.665	0.665	0.665	0.665	0.667	0.665	0.667	0.589	0.648	0.648	0.630
	95		0.664	0.667	0.666	0.666	0.670	0.666	0.666	0.667	0.616	0.651	0.654	0.660
US	10		0.541	0.567	0.523	0.514	0.558	0.559	0.568	0.560	0.499	0.530	0.510	0.511
	25		0.526	0.544	0.519	0.514	0.538	0.550	0.543	0.549	0.509	0.502	0.511	0.506
	50	$s = 0.236$	0.556	0.563	0.561	0.548	0.566	0.595	0.591	0.589	0.502	0.526	0.509	0.506
	75		0.605	0.594	0.606	0.598	0.600	0.637	0.637	0.635	0.507	0.507	0.545	0.556
	95		0.605	0.609	0.615	0.606	0.600	0.640	0.638	0.640	0.528	0.546	0.537	0.535
DS	10		0.529	0.548	0.510	0.514	0.549	0.551	0.552	0.548	0.509	0.515	0.516	0.518
	25		0.572	0.553	0.549	0.534	0.566	0.591	0.588	0.592	0.505	0.528	0.513	0.504
	50	$s = 0.236$	0.562	0.558	0.562	0.561	0.565	0.604	0.608	0.596	0.506	0.520	0.502	0.497
	75		0.573	0.567	0.592	0.577	0.583	0.622	0.617	0.617	0.499	0.524	0.508	0.495
	95		0.566	0.543	0.579	0.573	0.553	0.611	0.609	0.613	0.478	0.505	0.469	0.481
SYS	10		0.537	0.514	0.517	0.508	0.558	0.547	0.559	0.552	0.501	0.499	0.504	0.505
	25		0.587	0.579	0.565	0.547	0.597	0.617	0.609	0.621	0.497	0.497	0.506	0.509
	50	$d = 0.236$	0.602	0.593	0.607	0.592	0.610	0.640	0.638	0.630	0.522	0.553	0.542	0.557
	75		0.614	0.609	0.632	0.613	0.632	0.649	0.647	0.652	0.509	0.513	0.512	0.505
	95		0.611	0.609	0.617	0.606	0.620	0.644	0.641	0.642	0.502	0.521	0.516	0.511
CYC	10		0.512	0.517	0.505	0.505	0.518	0.514	0.508	0.514	0.501	0.504	0.512	0.514
	25		0.506	0.508	0.509	0.504	0.508	0.513	0.514	0.507	0.495	0.494	0.504	0.496
	50	$a = 0.097$	0.512	0.510	0.508	0.506	0.507	0.533	0.531	0.521	0.499	0.495	0.501	0.495
	75		0.504	0.508	0.508	0.502	0.506	0.505	0.507	0.506	0.501	0.501	0.506	0.505
	95		0.513	0.502	0.513	0.514	0.508	0.526	0.527	0.527	0.503	0.505	0.499	0.499

4.3. Results for Online Binary Classification with Real Data

We further evaluated the performance of our proposed CSPA algorithms against standard PA models using a real-world dataset from the pharmaceutical manufacturing industry, so called as the MSD Continuous Pharmaceutical Manufacturing Dataset, released as part of the 2024 MSOM Data-Driven Research Challenge [54]. The dataset consists of approximately 300 million samples collected over 120 hours of continuous tablet production, capturing real-time measurements of 75 process parameters. The dataset reflects a high-stakes industrial environment where early detection of abnormal patterns is essential for ensuring product quality and preventing costly production failures.

The dataset focuses on a continuous manufacturing (CM) process for pharmaceutical tablet production, which consists of four key stages: (1) formulation, (2) blending, (3) compression, and (4) coating. Unlike traditional batch manufacturing, CM allows for the uninterrupted flow of materials through each stage, significantly reducing production time and enhancing

overall efficiency. However, this continuous flow also presents challenges in real-time monitoring and control, as deviations in critical process parameters, such as temperature, humidity, or material flow rates, can quickly propagate throughout the system. The dataset captures both inline process data (e.g., mass flow rates, screw speeds, and compression forces) and contextual quality measurements (e.g., tablet weight, thickness, hardness, and drug content uniformity), offering a comprehensive representation of the production dynamics. For our analysis, we focused on two key process parameters from the blending stage: **Mass Flow Blender 1**, which measures the rate of material throughput during the initial mixing phase, and **OOS Concentration at Blender 2 Inlet**, which measures the percentage of material identified as out-of-specification entering from an upstream process. The characteristics of both datasets are presented in Table 11.

To prepare the MSD dataset for online learning, we applied a series of preprocessing steps. As the dataset lacked explicit labels, we analyzed the time series to identify anomalies through a rule-based labeling approach. First, the continuous time series was segmented into windows of length 10, consistent with the optimal performance range observed for the CSPA algorithms on the synthetic dataset. From each window, we extracted statistical and shape-based features, including mean, standard deviation, skewness, slope, and peak/valley counts. A window was labeled as abnormal if it met any of the following criteria: (1) standard deviation greater than 0.5 (indicating high volatility), (2) absolute slope greater than 1.0 (indicating a sharp trend), or (3) peak count exceeding 3 (suggesting erratic behavior). Windows not meeting any of these thresholds were labeled as normal. Visual inspection confirmed that the detected patterns corresponded to clear shifts in the data structure, consistent with abnormal behavior. Finally, the dataset was normalized and scaled to standardize feature values for model input.

As shown in Table 12, CSPA₁ and CSPA₂ outperform the standard PA algorithm across both datasets in terms of G-Mean, sensitivity, and specificity. Among the CSPA variants, CSPA₁ yields slightly higher sensitivity and G-Mean on the **Mass Flow Blender 1** dataset, whereas CSPA₂ performs marginally better on the **OOS Concentration at Blender 2 Inlet** dataset, which shows a greater degree of class imbalance (see Table 11). In contrast, the cost-sensitive variants CSPA- ℓ^I and CSPA- ℓ^{II} demonstrate lower performance, particularly in sensitivity and G-Mean. The standard OGD, CSOGD-I, and CSOGD-II show comparable results across both datasets, though their performance is not significantly different from the CSPA methods. Notably, the CSPA models are also more stable, as indicated by lower standard deviations in their performance metrics. In the **OOS** dataset, the high G-Mean scores are primarily driven by high sensitivity, which comes at the cost of relatively low specificity (around 50%). This indicates an increased false positive rate, an expected trade-off in industrial monitoring applications, where minimizing false negatives is often prioritized to ensure safety and process reliability.

Table 11: MSD data characteristic

MSD Dataset	# Samples	T_p	T_n	Imb
Mass Flow Blender 1	439,835	46,711	393,124	1:9.4
OOS Concentration at Blender 2 Inlet	439,835	567	439,268	1:775.7

Table 12: Comparative results from the MSD continuous pharmaceutical manufacturing dataset

Algorithm	Mass Flow Blender 1			OOS Concentration at Blender 2 Inlet		
	G-Mean(%)	Sensitivity(%)	Specificity(%)	G-Mean(%)	Sensitivity(%)	Specificity(%)
OGD [25]	56.736 $\pm$ 0.026	59.876 $\pm$ 0.039	53.760 $\pm$ 0.015	57.140 $\pm$ 0.042	65.178 $\pm$ 0.096	50.093 $\pm$ 0.001
CSOGD-I [32]	56.782 $\pm$ 0.023	59.944 $\pm$ 0.033	53.787 $\pm$ 0.013	57.147 $\pm$ 0.101	65.195 $\pm$ 0.229	50.093 $\pm$ 0.001
CSOGD-II [32]	56.718 $\pm$ 0.031	59.850 $\pm$ 0.046	53.750 $\pm$ 0.018	57.184 $\pm$ 0.092	65.279 $\pm$ 0.209	50.094 $\pm$ 0.001
PA [19]	50.804 $\pm$ 0.071	51.167 $\pm$ 0.104	50.444 $\pm$ 0.040	54.882 $\pm$ 0.461	60.177 $\pm$ 1.004	50.056 $\pm$ 0.005
PA ₁ [19]	56.189 $\pm$ 0.062	59.068 $\pm$ 0.092	53.451 $\pm$ 0.035	57.166 $\pm$ 0.089	65.238 $\pm$ 0.201	50.093 $\pm$ 0.001
PA ₂ [19]	54.105 $\pm$ 0.065	55.992 $\pm$ 0.096	52.281 $\pm$ 0.036	57.123 $\pm$ 0.183	65.141 $\pm$ 0.413	50.092 $\pm$ 0.003
CSPA ₁	56.942 $\pm$ 0.018	60.182 $\pm$ 0.027	53.877 $\pm$ 0.010	57.057 $\pm$ 0.045	64.992 $\pm$ 0.101	50.091 $\pm$ 0.001
CSPA ₂	56.903 $\pm$ 0.014	60.123 $\pm$ 0.021	53.855 $\pm$ 0.008	57.315 $\pm$ 0.138	65.575 $\pm$ 0.314	50.095 $\pm$ 0.002
CSPA- ℓ^I	51.841 $\pm$ 0.075	52.676 $\pm$ 0.109	51.019 $\pm$ 0.042	54.140 $\pm$ 0.335	58.572 $\pm$ 0.720	50.046 $\pm$ 0.004
CSPA- ℓ^{II}	50.812 $\pm$ 0.082	51.177 $\pm$ 0.119	50.448 $\pm$ 0.045	54.857 $\pm$ 0.278	60.121 $\pm$ 0.606	50.055 $\pm$ 0.003
CSPA ₁ - ℓ^I	56.889 $\pm$ 0.022	60.103 $\pm$ 0.032	53.847 $\pm$ 0.012	57.057 $\pm$ 0.025	64.992 $\pm$ 0.057	50.091 $\pm$ 0.000
CSPA ₁ - ℓ^{II}	56.941 $\pm$ 0.024	60.180 $\pm$ 0.036	53.876 $\pm$ 0.014	56.948 $\pm$ 0.349	64.749 $\pm$ 0.784	50.088 $\pm$ 0.006
CSPA ₂ - ℓ^I	56.813 $\pm$ 0.016	59.991 $\pm$ 0.023	53.804 $\pm$ 0.009	56.965 $\pm$ 0.192	64.786 $\pm$ 0.432	50.090 $\pm$ 0.004
CSPA ₂ - ℓ^{II}	56.854 $\pm$ 0.033	60.052 $\pm$ 0.048	53.827 $\pm$ 0.018	52.188 $\pm$ 0.529	54.453 $\pm$ 1.101	50.023 $\pm$ 0.006

5. Conclusion

This study introduces a novel cost-sensitive online learning framework for CCPR problems, addressing the critical challenge of class imbalance in real-time industrial monitoring. By extending the PA and OGD algorithms into cost-sensitive variants, CSPA and CSOGD, we incorporate class-specific penalties and modified loss functions to better detect rare but consequential anomalies. Our extensive experiments on synthetic and real-world datasets demonstrate that the proposed CSPA models outperform standard online learning methods, particularly in scenarios involving short observation windows or subtle abnormal patterns. The integration of both class-specific penalties and cost-sensitive loss functions yields the most robust and stable performance for online monitoring systems. While the framework shows strong generalizability across most pattern types, the stratification pattern remains a notable exception, which remained challenging for both linear CSPA and CSOGD algorithms. Incorporating nonlinear feature mappings (e.g., kernel-based models), or nonlinear models, such as Fourier Online Gradient Descent (FOGD) and Nyström Online Gradient Descent (NOGD) [62], is a possible direction for improving stratification pattern detection, which we leave for future investigation.

Future research should explore extending the proposed CSPA framework to multi-class CCPR scenarios, where multiple abnormal patterns may present. This would require addressing several key technical challenges: (1) reformulating the binary loss functions (e.g., hinge loss, ℓ^I , ℓ^{II}) into a multi-class, cost-sensitive loss; (2) defining a $K \times K$ misclassification cost matrix (C_{ij} , where K is the number of classes); and (3) re-deriving the closed-form CSPA update rules for the derived multi-class optimization problem. Integrating the cost-sensitive

online learning framework with deep learning models, such as the convolutional neural networks [58], could further enhance pattern recognition, especially in high-dimensional or non-linear data scenarios. Furthermore, adaptive tuning/optimizing the cost parameters will be critical for improving performance and further enhancing the applicability of cost-sensitive online learning in complex industrial environments.

6. Acknowledgment

This work was supported by the National Science Foundation under Grants No...

References

- [1] W. A. Shewhart, Quality control charts, *The Bell System Technical Journal* 5 (4) (1926) 593–603.
- [2] W. Hachicha, A. Ghorbel, A survey of control-chart pattern-recognition literature (1991–2010) based on a new conceptual classification scheme, *Computers & Industrial Engineering* 63 (1) (2012) 204–222.
- [3] M. Malindzakova, K. Čulková, J. Trpčevská, Shewhart control charts implementation for quality and production management, *Processes* 11 (4) (2023) 1246.
- [4] A. Addeh, A. Khormali, N. A. Golilarz, Control chart pattern recognition using rbf neural network with new training algorithm and practical features, *ISA Transactions* 79 (2018) 202–216.
- [5] R. Ünlü, A robust data simulation technique to improve early detection performance of a classifier in control chart pattern recognition systems, *Information Sciences* 548 (2021) 18–36.
- [6] E. Garcia, R. Penabaena-Niebles, M. Jubiz-Diaz, A. Perez-Tafur, Concurrent control chart pattern recognition: A systematic review, *Mathematics* 10 (6) (2022) 934.
- [7] L. Xue, H. Wu, H. Zheng, Z. He, Control chart pattern recognition for imbalanced data based on multi-feature fusion using convolutional neural network, *Computers & Industrial Engineering* 182 (2023) 109410.
- [8] P. Xanthopoulos, T. Razzaghi, A weighted support vector machine method for control chart pattern recognition, *Computers & Industrial Engineering* 70 (2014) 134–149.
- [9] P. H. Tran, A. Ahmadi Nadi, T. H. Nguyen, K. D. Tran, K. P. Tran, Application of machine learning in statistical process control charts: A survey and perspective, in: *Control charts and machine learning for anomaly detection in manufacturing*, Springer, 2022, pp. 7–42.
- [10] R.-S. Guh, F. Zorriassatine, J. Tannock, C. O’Brien, On-line control chart pattern detection and discrimination—a neural network approach, *Artificial Intelligence in Engineering* 13 (4) (1999) 413–425.

- [11] R.-S. Guh, A hybrid learning-based model for on-line detection and analysis of control chart patterns, *Computers & Industrial Engineering* 49 (1) (2005) 35–62.
- [12] S. Du, J. Lv, L. Xi, An integrated system for on-line intelligent monitoring and identifying process variability and its application, *International Journal of Computer Integrated Manufacturing* 23 (6) (2010) 529–542.
- [13] P. Malaca, L. F. Rocha, D. Gomes, J. Silva, G. Veiga, Online inspection system based on machine learning techniques: real case study of fabric textures classification for the automotive industry, *Journal of Intelligent Manufacturing* 30 (2019) 351–361.
- [14] T. Zan, Z. Liu, H. Wang, M. Wang, X. Gao, Control chart pattern recognition using the convolutional neural network, *Journal of Intelligent Manufacturing* 31 (2020) 703–716.
- [15] J.-E. Chiu, C.-H. Tsai, On-line concurrent control chart pattern recognition using singular spectrum analysis and random forest, *Computers & Industrial Engineering* 159 (2021) 107538.
- [16] J.-E. Chiu, S.-Z. Fu, On-line recognition of mixture control chart patterns using hybrid CNN and LSTM for auto-correlated processes, *Computers & Industrial Engineering* 198 (2024) 110674.
- [17] D. T. Pham, A. Chan, Unsupervised adaptive resonance theory neural networks for control chart pattern recognition, *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture* 215 (1) (2001) 59–67.
- [18] S. Haghtalab, P. Xanthopoulos, K. Madani, A robust unsupervised consensus control chart pattern recognition framework, *Expert Systems with Applications* 42 (19) (2015) 6767–6776.
- [19] K. Crammer, O. Dekel, J. Keshet, S. Shalev-Shwartz, Y. Singer, M. K. Warmuth, Online passive-aggressive algorithms., *Journal of Machine Learning Research* 7 (3) (2006).
- [20] W.-A. Yang, W. Zhou, Autoregressive coefficient-invariant control chart pattern recognition in autocorrelated manufacturing processes using neural network ensemble, *Journal of Intelligent Manufacturing* 26 (2015) 1161–1180.
- [21] S.-Y. Lin, R.-S. Guh, Y.-R. Shiue, Effective recognition of control chart patterns in autocorrelated data using a support vector machine based approach, *Computers & Industrial Engineering* 61 (4) (2011) 1123–1134.
- [22] S. Du, D. Huang, J. Lv, Recognition of concurrent control chart patterns using wavelet transform decomposition and multiclass support vector machines, *Computers & Industrial Engineering* 66 (4) (2013) 683–695.
- [23] Y. Li, P. Long, The relaxed online maximum margin algorithm, *Advances in Neural Information Processing Systems* 12 (1999).

- [24] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain., *Psychological Review* 65 (6) (1958) 386.
- [25] M. Zinkevich, Online convex programming and generalized infinitesimal gradient ascent, in: *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*, 2003, pp. 928–936.
- [26] A. A. Alashhab, M. S. Zahid, B. Isyaku, A. A. Elnour, W. Nagmeldin, A. Abdelmaboud, T. A. A. Abdullah, U. D. Maiwada, Enhancing DDoS attack detection and mitigation in SDN using an ensemble online machine learning model, *IEEE Access* 12 (2024) 51630–51649.
- [27] S. Mahjoubi, R. Barhemat, W. Meng, Y. Bao, AI-guided auto-discovery of low-carbon cost-effective ultra-high performance concrete (UHPC), *Resources, Conservation and Recycling* 189 (2023) 106741.
- [28] S. Saleh, B. Cherradi, O. El Gannour, S. Hamida, O. Bouattane, Predicting patients with parkinson’s disease using machine learning and ensemble voting technique, *Multimedia Tools and Applications* 83 (11) (2024) 33207–33234.
- [29] S. C. Hoi, D. Sahoo, J. Lu, P. Zhao, Online learning: A comprehensive survey, *Neurocomputing* 459 (2021) 249–289.
- [30] M. Bag, S. K. Gauri, S. Chakraborty, An expert system for control chart pattern recognition, *The International Journal of Advanced Manufacturing Technology* 62 (2012) 291–301.
- [31] N. Japkowicz, S. Stephen, The class imbalance problem: A systematic study, *Intelligent Data Analysis* 6 (5) (2002) 429–449.
- [32] J. Wang, P. Zhao, S. C. Hoi, Cost-sensitive online classification, *IEEE Transactions on Knowledge and Data Engineering* 26 (10) (2013) 2425–2438.
- [33] P. Zhao, Y. Zhang, M. Wu, S. C. Hoi, M. Tan, J. Huang, Adaptive cost-sensitive online classification, *IEEE Transactions on Knowledge and Data Engineering* 31 (2) (2018) 214–228.
- [34] Z. Chen, Z. Fang, W. Fan, A. Edwards, K. Zhang, CSTG: An effective framework for cost-sensitive sparse online learning, in: *Proceedings of the 2017 SIAM International Conference on Data Mining*, SIAM, 2017, pp. 759–767.
- [35] Z. Chen, Z. Fang, V. Sheng, A. Edwards, K. Zhang, CSRDA: Cost-sensitive regularized dual averaging for handling imbalanced and high-dimensional streaming data, in: *2021 IEEE International Conference on Big Knowledge (ICBK)*, IEEE, 2021, pp. 164–173.
- [36] Y. Chen, Z. Hong, X. Yang, Cost-sensitive online adaptive kernel learning for large-scale imbalanced classification, *IEEE Transactions on Knowledge and Data Engineering* 35 (10) (2023) 10554–10568.

- [37] P. Okafor, Bioprocess monitoring in the biopharmaceutical industry using filtering and online cost-sensitive supervised learning, Master’s thesis, University of Oklahoma–Graduate College (2024).
- [38] L. Xiao, Dual averaging method for regularized stochastic learning and online optimization, *Advances in Neural Information Processing Systems* 22 (2009).
- [39] Z. Chen, V. Sheng, A. Edwards, K. Zhang, An effective cost-sensitive sparse online learning framework for imbalanced streaming data classification and its application to online anomaly detection, *Knowledge and Information Systems* 65 (1) (2023) 59–87.
- [40] Z. Chen, H. Zhan, V. Sheng, A. Edwards, K. Zhang, Proximal cost-sensitive sparse group online learning, in: *2022 IEEE International Conference on Big Data (Big Data)*, IEEE, 2022, pp. 495–504.
- [41] J. Langford, L. Li, T. Zhang, Sparse online learning via truncated gradient, *Advances in Neural Information Processing Systems* 21 (2008).
- [42] Y. Chen, X. Yang, H.-L. Dai, Cost-sensitive continuous ensemble kernel learning for imbalanced data streams with concept drift, *Knowledge-Based Systems* 284 (2024) 111272.
- [43] M. Usman, H. Chen, Intensive class imbalance learning in drifting data streams, *IEEE Transactions on Emerging Topics in Computational Intelligence* 8 (5) (2024) 3503–3517.
- [44] Y. Li, J. Jin, J. Ma, F. Zhu, B. Jin, J. Liang, C. P. Chen, Imbalanced least squares regression with adaptive weight learning, *Information Sciences* 648 (2023) 119541.
- [45] Y. Li, J. Jin, H. Tao, Y. Xiao, J. Liang, C. P. Chen, Complemented subspace-based weighted collaborative representation model for imbalanced learning, *Applied Soft Computing* 153 (2024) 111319.
- [46] Y. Li, Y. Wang, J. Jin, W. Zhang, H. Tao, H. Wu, C. P. Chen, Imbalanced broad learning system with label relaxation and sample weight adaptation, *Applied Soft Computing* 182 (2025) 113543.
- [47] A. Carnero, C. Martín, G. Jeon, M. Díaz, Online learning and continuous model upgrading with data streams through the Kafka-ML framework, *Future Generation Computer Systems* (2024).
- [48] S. C. Hoi, J. Wang, P. Zhao, Libol: A library for online learning algorithms, *Journal of Machine Learning Research* 15 (1) (2014) 495.
- [49] P. Zhao, D. Wang, P. Wu, S. C. Hoi, A unified framework for sparse online learning, *ACM Transactions on Knowledge Discovery from Data (TKDD)* 14 (5) (2020) 1–20.
- [50] Z. Chen, V. Sheng, A. Edwards, K. Zhang, Cost-sensitive sparse group online learning for imbalanced data streams, *Machine Learning* 113 (7) (2024) 4407–4444.

- [51] Y. Freund, R. E. Schapire, Large margin classification using the perceptron algorithm, in: Proceedings of the Eleventh Annual Conference on Computational Learning Theory, 1998, pp. 209–217.
- [52] J. Wang, P. Zhao, S. C. Hoi, Exact soft confidence-weighted learning, arXiv preprint arXiv:1206.4612 (2012).
- [53] R. Ünlü, Cost-oriented LSTM methods for possible expansion of control charting signals, Computers & Industrial Engineering 154 (2021) 107163.
- [54] T. Martagan, M. Baaijens, C. Dirckx, J. Holman, R. Meyer, O. Repping, B. van Ravenstein, MSD: Continuous pharmaceutical manufacturing data for the 2024 MSOM data-driven research challenge, Manufacturing & Service Operations Management (2024).
- [55] P. S. Foundation, Concurrent.futures — Launching parallel tasks, <https://docs.python.org/3/library/concurrent.futures.html>, Accessed: 2025-06-30 (2024).
- [56] P. S. Foundation, Subprocess management, <https://docs.python.org/3/library/subprocess.html>, Accessed: 2025-06-30 (2024).
- [57] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2019, pp. 2623–2631.
- [58] D. Fuqua, T. Razzaghi, A cost-sensitive convolution neural network learning for control chart pattern recognition, Expert Systems with Applications 150 (2020) 113275.
- [59] M. Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, Journal of the American Statistical Association 32 (200) (1937) 675–701.
- [60] P. B. Nemenyi, Distribution-free multiple comparisons., Princeton University, 1963.
- [61] J. Demšar, Statistical comparisons of classifiers over multiple data sets, Journal of Machine Learning Research 7 (Jan) (2006) 1–30.
- [62] J. Lu, S. C. Hoi, J. Wang, P. Zhao, Z.-Y. Liu, Large scale online kernel learning, Journal of Machine Learning Research 17 (47) (2016) 1–43.